



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У
НОВОМ САДУ



Лазар Нагулов

DSL и LSP за генерисање тест података

ЗАВРШНИ РАД

Основне академске студије

Нови Сад, 2026

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА 21000 НОВИ САД, Трг Доситеја Обрадовића 6	Број:
	ЗАДАТАК ЗА ЗАВРШНИ РАД	Датум:

(Податке уноси предметни наставник - ментор)

Студијски програм:	Софтверско инжењерство и информационе технологије		
Студент:	Лазар Нагулов	Број индекса:	SV61/2022
Степен и врста студија:	Основне академске студије		
Област:	Електротехничко и рачунарско инжењерство		
Ментор:	Игор Дејановић		
НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА: - проблем – тема рада; - начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;			

НАСЛОВ ЗАВРШНОГ РАДА:

DSL и LSP за генерисање тест података

ТЕКСТ ЗАДАТКА:

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Quia ipsum suspendisse ultrices gravida. Risus commodo viverra maecenas accumsan.
--

Руководилац студијског програма:	Ментор рада:

Примерак за: □ - Студента; □ - Ментора
--



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР:	
Идентификациони број, ИБР:	
Тип документације, ТД:	Монографска документација
Тип записа, ТЗ:	Текстуални штампани материјал
Врста рада, ВР:	Дипломски - бечелор рад
Аутор, АУ:	Лазар Нагулов
Ментор, МН:	Др Игор Дејановић, редовни професор
Наслов рада, НР:	DSL и LSP за генерисање тест података
Језик публикације, ЈП:	српски/ћирилица
Језик извода, ЈИ:	српски/енглески
Земља публикавања, ЗП:	Република Србија
Уже географско подручје, УГП:	Војводина
Година, ГО:	2026
Издавач, ИЗ:	Ауторски репринт
Место и адреса, МА:	Нови Сад, трг Доситеја Обрадовића 6
Физички опис рада, ФО: (поглавља/страна/ цитата/табела/слика/графика/прилога)	8/60/40/6/38/0/2
Научна област, НО:	Електротехничко и рачунарско инжењерство
Научна дисциплина, НД:	Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, ПО:	Шаблон, завршни рад, упутство
УДК	
Чува се, ЧУ:	У библиотеци Факултета техничких наука, Нови Сад
Важна напомена, ВН:	
Извод, ИЗ:	Овај документ представља упутство за писање завршних радова на Факултету техничких наука Универзитета у Новом Саду. У исто време је и шаблон за Typst.
Датум прихватања теме, ДП:	
Датум одбране, ДО:	01.01.2025
Чланови комисије, КО:	Председник: Др Петар Петровић, ванредни професор
	Члан: Др Марко Марковић, доцент
	Члан:
Члан, ментор:	Др Игор Дејановић, редовни професор
	Потпис ментора



UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES
21000 NOVI SAD, Trg Dositeja Obradovića 6

KEY WORDS DOCUMENTATION

Accession number, ANO :	
Identification number, INO :	
Document type, DT :	Monographic publication
Type of record, TR :	Textual printed material
Contents code, CC :	
Author, AU :	Lazar Nagulov
Mentor, MN :	Igor Dejanović, Phd., full professor
Title, TI :	DSL and LSP for test data generation
Language of text, LT :	Serbian
Language of abstract, LA :	Serbian/English
Country of publication, CP :	Republic of Serbia
Locality of publication, LP :	Vojvodina
Publication year, PY :	2026
Publisher, PB :	Author's reprint
Publication place, PP :	Novi Sad, Dositeja Obradovica sq. 6
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	8/60/40/6/38/0/2
Scientific field, SF :	Electrical and Computer Engineering
Scientific discipline, SD :	Applied computer science and informatics
Subject/Key words, S/KW :	Template, thesis, tutorial
UC	
Holding data, HD :	The Library of Faculty of Technical Sciences, Novi Sad
Note, N :	
Abstract, AB :	This document provides guidelines for writing final theses at the Faculty of Technical Sciences, University of Novi Sad. At the same time, it serves as a Typst template.
Accepted by the Scientific Board on, ASB :	
Defended on, DE :	01.01.2025
Defended Board, DB :	President: Petar Petrović, Phd., assoc. professor
	Member: Marko Marković, Phd., asist. professor
	Member:
Member, Mentor:	Igor Dejanović, Phd., full professor
	Mentor's sign



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

Садржај

1	Увод	1
1.1	Мотивација	1
1.2	Предмет и цињ рада	1
1.3	Допринос рада	1
1.4	Структура рада	1
2	Теоријске основе	3
2.1	Језици специфични за домен	3
2.2	Фазе обраде програмског језика	4
2.3	Међурепрезентације	5
2.4	Систем модула	7
2.5	Генерисање података	9
2.6	Протокол језичких сервера	11
3	Анализа постојећих решења	13
3.1	Језици специфични за домен	13
3.2	Системи за дефинисање шема	14
3.3	Системи за генерисање тест података	15
3.4	Закључна разматрања и упоредни приказ	16
4	Дизајн решења	17
4.1	Архитектура система	17
4.2	Дизајн језика специфичног за домен	18
4.3	Дизајн семантичке анализе	20
4.4	Дизајн високонивојске међурепрезентације	20
4.5	Дизајн модула	21
4.6	Дизајн генератора података	22
4.7	Дизајн језичког сервера	22
4.8	Дизајн командног интерфејса	25
5	Имплементација	27
5.1	Лексичка анализа	27
5.2	Синтаксна анализа	27
5.3	Апстрактно синтаксно стабло	29
5.4	Семантичка анализа	29
5.5	Високонивојска међурепрезентација	32
5.6	Систем модула	32
5.7	Генерисање података	34
5.8	Језички сервер	34
5.9	Командни интерфејс	35
6	Евалуација и дискусија	37
6.1	Експериментално окружење	37
6.2	Евалуација процеса превођења	37
6.3	Евалуација система модула	38
6.4	Евалуација генерисања података	39
6.5	Евалуација језичког сервера	41
6.6	Дискусија резултата	42
7	Закључак	43

8 Будући рад	45
8.1 Унапређење система компилације	45
8.2 Средњенивојска међурепрезентација	45
Списак слика	47
Списак листинга	49
Списак табела	51
Списак коришћених скраћеница	53
Списак коришћених појмова	55
Биографија	57
Литература	59

Развој софтвера захтева континуирано тестирање. Језици специфични за домен (енг. *Domain Specific Language*, DSL) олакшавају решавање проблема у конкретној области [1]. Развој подршке за интегрисана развојна окружења захтева значајне ресурсе [2]. Протокола језичких сервера (енг. *Language Server Protocol*, LSP) стандардизује комуникацију између едитора и алата за анализу кода [3]. Стандард раздваја језички неутралан *editor* и језички сервер [4].

1.1 Мотивација

Креирање посебних података за сваки едитор повећава трошкове развоја [5]. Недостатак алата је чест разлог за одбацивање DSL-а [6]. Стандардизација путем LSP-а омогућава јединствену имплементацију подршке [4]. Програмери добијају конзистентно искуство независно од изабраног алата. Постојећи алати за генерацију података захтевају писање императивног кода или коришћење визуелних интерфејса. Декларативни приступ поједностављује процес дефинисања структуре података.

1.2 Предмет и циљ рада

Предмет рада представља дизајн и имплементација DSL-а за генерисање тест података. Рад обухвата развој преводилаца и пратећег језичког сервера. Циљ рада је унапређење ефикасности програмера током процеса развоја софтвера. Систем треба да омогући детекцију грешака у реалном времену. Алат треба да обезбеди навигацију кроз дефиниције симбола и аутоматско довршавање кода.

1.3 Допринос рада

Рад представља нову декларативну синтаксу за опис модела података. Систем уводи модуларну архитектуру. Модуларност убрзава процес компилације пројеката. Имплементирани језички сервер пружа подршку развојним окружењима. Решење обједињује брзину локалног извршавања са функционалностима анализе кода.

1.4 Структура рада

Рад садржи осам поглавља. Друго поглавље описује теоријске основе језичких сервера и преводилаца. Треће поглавље анализира постојећа решења у области генерације података. Четврто поглавље приказује дизајн предложеног система. Пето поглавље детаљно описује техничку имплементацију решења. Шесто поглавље представља резултате тестирања и евалуацију перформанси. Седмо поглавље доноси закључак, док осмо поглавље разматра могуће правце будућег рада.

Ово поглавље приказује теоријске концепте неопходне за разумевање архитектуре и имплементације система. Први део дефинише DSL и њихову класификацију (поглавље 2.1). Други део анализира фазе лексичке, синтаксне и семантичке анализе текста (поглавље 2.2). Трећи део дефинише концепте међуре-презентација у компајлерима (поглавље 2.3). Четврти део описује теоријске основе модуларних система (поглавље 2.4), док је пети сегмент посвећен алгоритмима за стохастичко генерисање података (поглавље 2.5). Напослетку, шести део описује архитектуру LSP-а и пратећих стандарда (поглавље 2.6).

2.1 Језици специфични за домен

DSL је програмски језик ограничених могућности намењен решавању проблема у конкретној области [1]. Синтакса и семантика DSL-а прилагођене су конкретном домену [1]. Дефиниција обухвата четири елемента:

- Први елемент чини природа рачунарског програмског језика за давање инструкција рачунару [1]. Структура језика омогућава људима разумевање изворног кода уз задржавање извршности на рачунару [1].
- Други елемент представља флуидност изражавања кроз међусобно комбиновање појединачних језичких конструкција [1].
- Трећи елемент је ограничена експресивност у односу на језике опште намене (енг. *General Purpose Language*, GPL) [1]. Ограничена експресивност подразумева подршку само за минималан скуп функционалности неопходних за домен [1]. Језик онемогућава развој целокупног софтверског система већ решава само један сегмент [1].
- Четврти елемент обухвата јасан фокус на уску област примене [1]. Фокус на домен произилази директно из ограничене могућности изражавања језика [1].

Инжењерство програмских језика прави разлику између DSL и GPL [6]. GPL пружају широк спектар могућности за решавање произвољних софтверских проблема [6]. DSL међају општу употребљивост за висок ниво апстракције и већу продуктивност унутар изабране области [6].

2.1.1 Класификација језика специфичних за домен

Разликују се интерни (енг. *internal*) и екстерни (енг. *external*) доменски језици [1], [6].

1. Интерни доменски језици реализују се унутар постојећег језика домаћина [1]. Синтакса интерних језика дефинисана је и ограничена синтаксним правилима изабраног језика домаћина [6].
2. Екстерни доменски језици поседују независну синтаксу у односу на друге програмске језике [1]. Развој екстерног језика захтева имплементацију засебног лексичког и синтаксног анализатора за анализу изворног текста [6]. Контрола над синтаксом омогућава креирање оптималних текстуалних конструкција за крајњег корисника [6].

2.1.2 Предности и изазови примене

Примена доменских језика доноси предности у процесу развоја софтвера [6]. Висок ниво апстракције омогућава доменским експертима боље разумевање и лакшу валидацију програмског кода [1]. Употреба специјализованих термина смањује број семантичких грешака и убрзава писање спецификација [6]. Развој доменских језика ствара специфичне инжењерске изазове [6]. Креирање екстерног језика подиже комплексност почетне софтверске архитектуре [1]. Доменски језик захтева изградњу и одржавање пратећих алата попут компајлера, језичких сервера и развојних окружења [6].

2.2 Фазе обраде програмског језика

Обрада доменског језика обухвата низ сукцесивних фаза које превode изворни код у међурепрезентацију спремну за извршавање [7]. Процес се дели на предњи део (енг. *frontend*) и задњи део (енг. *backend*) превода. Предњи део анализира изворни текст и утврђује исправност структуралних и семантичких правила [8]. Задњи део користи припремљене структуре података за покретање логике извршавања или генерисање циљног кода [1]. Имплементирани предњи део обезбеђује заједничку основу за механизам генерације тест података и језички сервер.

2.2.1 Лексичка анализа

Лексичка анализа представља почетну фазу у процесу обраде изворног кода [7]. Лексички анализатор (енг. *lexer*) прихвата низ карактера из текстуалне датотеке и групише карактере у логичке целине [9]. Логичке целине називају се лексеме [7]. Свака лексема се мапира у одговарајући токен који садржи тип и вредност препознатих карактера [9]. Токени представљају кључне речи, идентификаторе, литерале и операторе језика [8]. Лексички анализатор истовремено филтрира сувишне елементе изворног кода попут размака и коментара [9]. Резултат фазе је низ токена спреман за даљу синтаксну проверу.

2.2.2 Синтаксна анализа

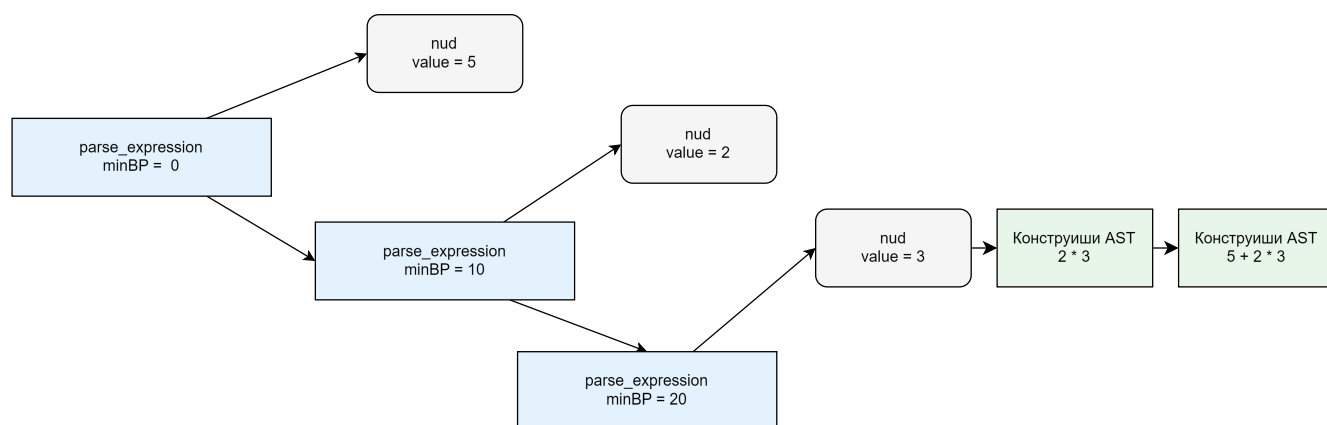
Синтаксна анализа утврђује структуралну исправност низа токена на основу дефинисане граматике језика [7]. Синтаксни анализатор (енг. *parser*) организује примљене токене у хијерархијску структуру података [8]. Хијерархијска структура назива се апстрактно синтаксно стабло (енг. *Abstract Syntax Tree, AST*) [9]. AST приказује операторе и операнде унутар сложених језичких конструкција [8]. Грешке откривене у процесу анализе се прослеђују кориснику или развојном окружењу путем језичког сервера.

2.2.2.1 Пратов синтаксни анализатор

Пратов синтаксни анализатор (енг. *Pratt parser*) је алгоритам синтаксне анализе заснован на првенству оператора одозго на доле [9]. Рад алгоритма почива на нумеричкој вредности која се назива снага везивања (енг. *binding power, BP*). BP експлицитно одређује ниво приоритета сваког оператора у језику [9]. Токенски елементи се мапирају у две функције за обраду изворног текста:

- префиксну функцију (енг. *null denotation* или *nud*), која обрађује токене на самом почетку израза и
- инфиксну функцију (енг. *left denotation* или *led*), која обрађује токене који се појављују са десне стране већ делимично изграђеног стабла [9].

Главна петља алгоритма пореди BP тренутног оператора са BP-ом следећег токена у низу [9]. Синтаксни анализатор наставља са груписањем аргумената у подстабло докле год је BP следећег токена већа од тренутне вредности [9]. Предност приступа је једноставност одржавања програмског кода за анализу математичких операција [8]. Визуелни приказ трансформације линеарног низа токена у хијерархијску структуру на основу BP-а приказан је на слици 1.



Слика 1: Трансформација израза $5 + 2 * 3$ у AST на основу BP-а оператора

2.2.3 Семантичка анализа

Семантичка анализа проверава да ли изворни код поштује правила значења и контекста програмског језика [7]. Фаза користи AST за доношење одлука о исправности програма [8]. Основне активности обухватају проверу компатибилности типова података и дефинисаност коришћених идентификатора [7]. Систем мапира откривене променљиве у структуре података које се називају табеле симбола (енг. *symbol table*) [8]. Табела симбола чува информације о опсегу видљивости, типовима и особинама сваког симбола [8]. Успешна семантичка анализа потврђује исправност улазних спецификација пре покретања алгорита за генерисање тест података [6].

2.3 Међурепрезентације

Међурепрезентација (енг. *Intermediate Representation*, IR) представља интерну структуру података коју преводилац користи за моделовање изворног програма између почетних и завршних фаза обраде [7]. Језички системи ретко врше директно превођење изворног кода у машински или извршни облик. Уместо тога, користи се више сукцесивних нивоа апстракције како би се логика синтаксне анализе раздвојила од семантичке провере, независне оптимизације и коначне генерације кода [10]. Зависно од блискости изворном коду или циљној архитектури, IR се категоризује у високонивојски (енг. *High-level Intermediate Representation*, HIR), средњенивојски (енг. *Medium-level Intermediate Representation*, MIR) и нисконивојски (енг. *Low-level Intermediate Representation*, LIR) [10]. Сваки слој је прилагођен специфичним захтевима фазе извршавања. Развој решења ставља фокус на семантичку анализу унутар HIR слоја, док имплементација MIR и LIR структура представља путању будућег проширења система.

2.3.1 Апстрактно синтаксно стабло

AST чини примарни слој који осликава граматичку структуру изворног текста [8]. AST је резултат фазе синтаксне анализе. За разлику од конкретног синтаксног стабла, AST издваја декоративне елементе језика као што су заграде, тачке и белине [9]. Чворови унутар структуре представљају операторе, семантичке конструкте и језичке директиве, док гране воде ка припадајућим операндима. Особине AST-а пружају основу за почетне фазе синтаксне провере и мапирање локација грешака. Директна повезаност са грама-тиком чини AST неефикасним за претраге глобалних веза.

2.3.2 Интернирање стрингова

Током синтаксне анализе изворног кода, преводилац се сусреће са појавом истих текстуалних идентификатора. Чување и сукцесивно поређење стрингова у сваком чвору IR-а уводи меморијски и рачунарски трошак. Превазилажење трошка постиже се техником интернирања стрингова (енг. *string interning*) [11].

Интернирање стрингова оптимизује меморију кроз имплементацију централизованог, непроменљивог базена стрингова (енг. *string pool*). Сваки стринг се уписује у базен само једном, при чему му се додељује нумерички идентификатор (енг. *symbol* или *ID*). Фазе унутар компајлерске архитектуре уместо комплетних текстуалних података користе додељене нумеричке кључеве. Замена операција поређења текстуалних низова поређењем целих бројева убрзава семантичку анализу и изградњу форми IR [7].

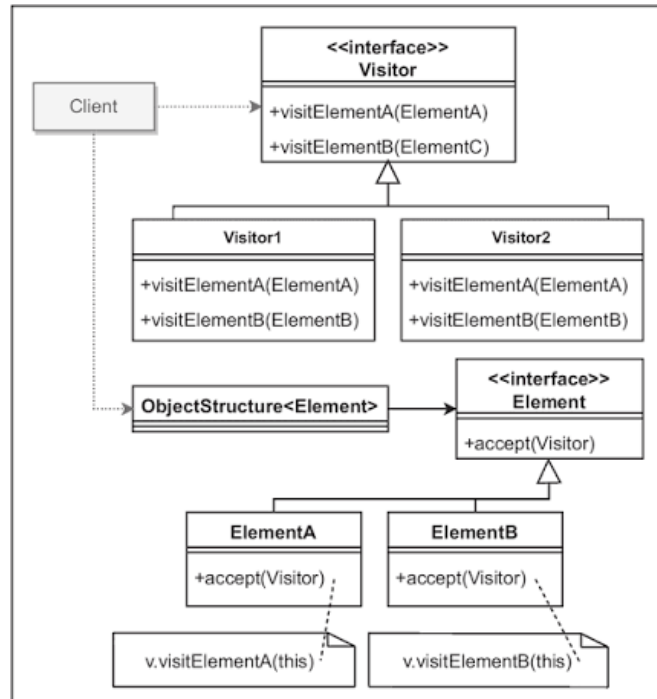
2.3.3 Дизајнерски шаблон посетилац

Дизајнерски шаблони су вишекратно употребљива решења за учестале проблеме у пројектовању софтвера [12]. У архитектури преводилаца, изазов представља проширење функционалности система без модификације класа [13]. За решавање проблема проширења функционалности користи се шаблон посетилац (енг. *Visitor*). Посетилац припада категорији образаца понашања [12].

Намена посетилаца је раздвајање алгорита од структуре података над којом се оперише [12]. У језичким процесорима, структура података је представљена чворовима AST-а [13]. Логика за семантичку анализу, оптимизацију или серијализацију локализује се унутар засебних класа, уместо да се уграђује директно у чворове стабла [13].

Посетилац се заснива на механизму двоструког упућивања (енг. *double dispatch*) [12]. Двоструко упућивање зависи истовремено од типа конкретног објекта посетилаца и од типа конкретног елемента структуре [12]. Структура посетилаца (слика 2) обухвата две компоненте:

- апстракцију елемента (енг. *Element*) која дефинише интерфејс са методом *accept(v: Visitor)* за преусмеравање позива конкретних чворова и
- апстракцију посетилаца која дефинише *visit* методе за извршавање алгоритама над конкретним класама елемената [12].



Слика 2: Структура дизајнерског шаблона посетилац [12]

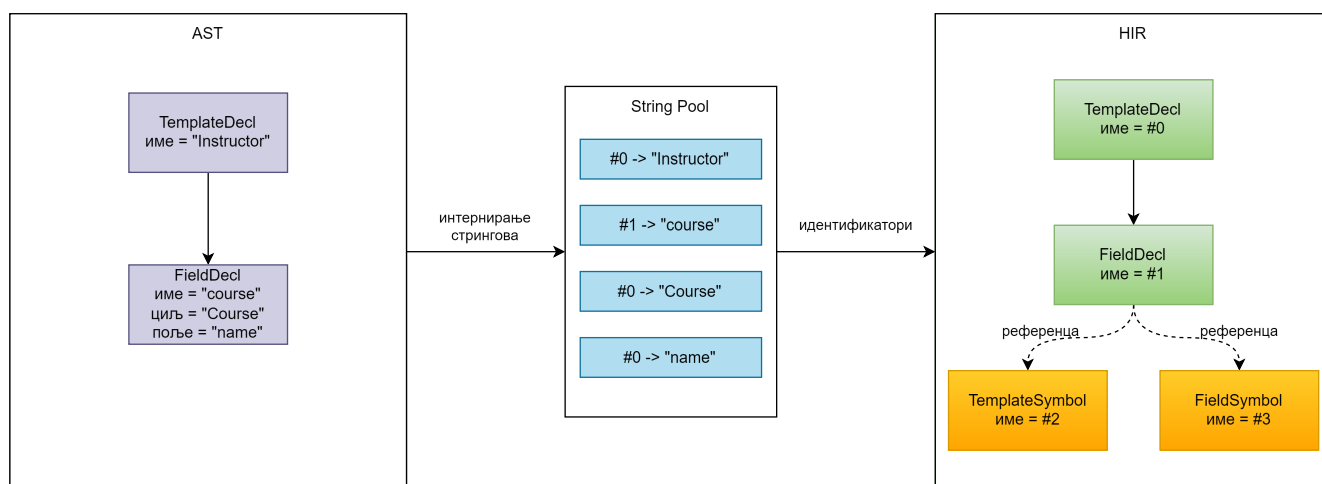
Посетилац се примењује када су испуњени услови да:

- објектна структура садржи елементе различитих класа са специфичним операцијама за сваку конкретну класу,
- постоји потреба за извршавањем несродних операција над елементима без додавања нових метода у класе структуре, те да
- се хијерархија класа објектне структуре ретко мења услед учесталог додавања нових операција над структуром [12].

Граматика развијеног језика је стабилна [13]. Синтаксна правила су фиксирана [13]. Структура чворова AST-а је због тога непроменљива [13]. Фиксирана хијерархија спречава потребу за накнадним модификацијама постојећих посетилаца. Посетилац омогућава додавање компајлерских пролаза кроз засебне компоненте без измене кода стабла. Примена посетилаца дозвољава независну имплементацију трансформације у HIR [11] и серијализацију података [14].

2.3.4 Високонивојска међурепрезентација

HIR представља корак апстракције у коме се текстуална структура програма трансформише у оријентисани граф релација [11]. Слика 3 приказује разлику између AST-а и HIR-а на примеру једне декларације шаблона. Док AST чува текстуалне идентификаторе и прати синтаксичку структуру програма, HIR користи интерниране идентификаторе и експлицитне везе ка симболима, чиме моделује семантичке односе између конструката. Изградња HIR-а отпочиње након валидације AST-а. Трансформација се заснива на процесу спуштања (енг. *lowering*). Спуштање елиминисе преостале синтаксне декорације и своди изразе на каноичке облике [10].



Слика 3: Поређење AST-а и HIR-а на примеру декларације шаблона.

Улога HIR-а је разрешавање референци, типова и међусобних односа између симбола [7]. HIR интегрише локалне симболе тренутног документа са глобалним симболима који су учитани из модула. Текстуални идентификатори су у фази спуштања замењени интернираним нумеричким кључевима. Фокус се тиме преусмерава са начина на који је код написан на семантичко значење и међусобну повезаност објеката. Компактност релационог графа чини HIR погодним за дуготрајно чување, поновно учитавање модуларних целина и генерисање излазних података.

2.4 Систем модула

Развој језичких система захтева декомпозицију спецификација на мање, независне логичке целине. Концепт модуларности почива на дефинисању изолованих компајлационих јединица (енг. *compilation units*) које међусобно комуницирају путем прецизно одређених интерфејса [6]. Подршка за модуле унутар преводиоца подразумева стриктно раздвајање локалних опсега видљивости (енг. *scope*) од увезених, спољних симбола. Имплементација система модула намеће потребу за трајним чувањем и дељењем међурезултата компајлације на нивоу датотечног система.

2.4.1 Простори имена и глобално адресирање

Повезивање више изворних датотека у програм доводи до колизије симбола. Колизија настаје када два независна модула дефинишу ентитете са истим текстуалним називом. Проблем колизије се решава увођењем простора имена (енг. *namespace*). Простор имена је изоловани контекст у којем сваки идентификатор мора бити јединствен [7]. Унутар архитектуре компајлера, имплементација простора имена захтева транзицију са локалног на глобално адресирање [10]. Глобално адресирање се реализује путем композитних кључева који обједињују идентификатор простора имена (модула) и локални идентификатор ентитета [11]. Примена композитних кључева обезбеђује униформну обраду свих симбола у меморији, независно од њиховог физичког порекла.

2.4.2 Повезивање модула и релокација

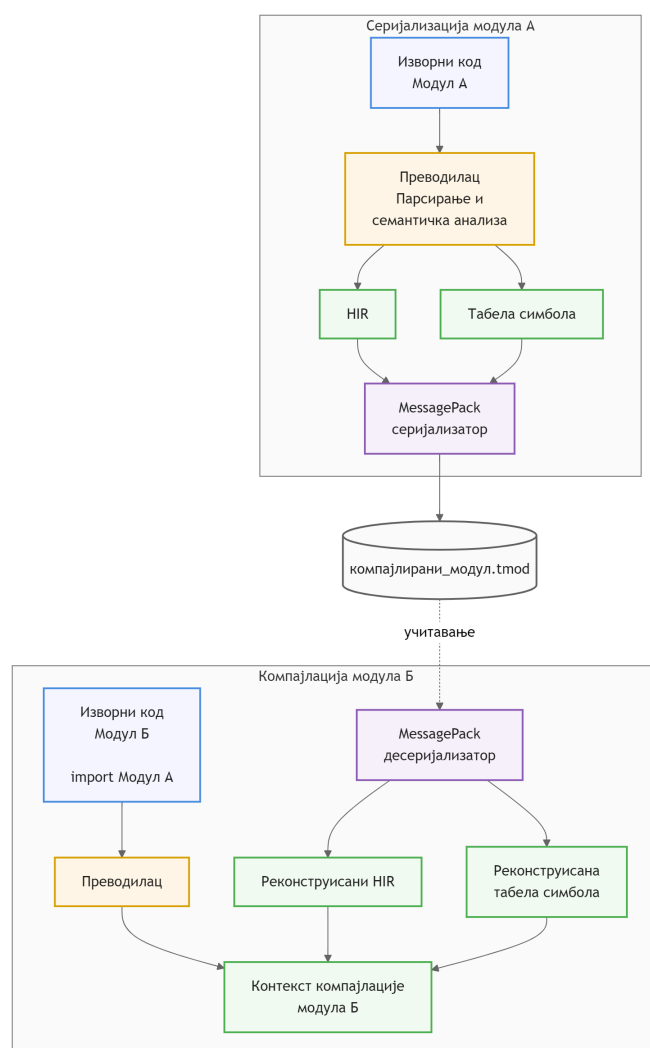
Независно преводиоње модула резултује стварањем модула који не познају коначни адресни простор програма. Идентификатори унутар модула су релативни и важе искључиво у контексту тог модула [10]. Процес спајања независних модула у извршну целину назива се повезивање (енг. *linking*) [7]. Релокација (енг. *relocation*) је корак током повезивања који ажурира нумеричке референце и меморијске адресе унутар учитаног кода (табела 1) [7]. Учитавање спољног модула захтева транслацију релативних идентификатора у вредности које одговарају тренутном глобалном стању система [11]. Транслација спречава преклапање адреса између увезених библиотека и обезбеђује конзистентност семантичког графа програма.

Табела 1: Пресликавање локалних идентификатора у глобални адресни простор током релокације.

Ентитет	Стање на диску	Стање у меморији након релокације
Структура_А	LocalId(1) унутар зависности	GlobalId(ModuleId(4), LocalId(1))
Шаблон_В	LocalId(2) унутар зависности	GlobalId(ModuleId(4), LocalId(2))
Локална променљива	LocalId(1) у тренутном фајлу	GlobalId(ModuleId(0), LocalId(1))

2.4.3 Бинарна серијализација и формати без шеме

Подаци се унутар процеса чувају у меморијским структурама. За трајно складиштење ових структура на диску или за њихов пренос кроз мрежу, потребно је претворити податке из меморије у линеарни низ бајтова. Поступак претварања података у линеарни низ бајтова се назива серијализација (енг. *serialization* или *encoding*) [15]. Серијализовани запис HIR-а и пратеће таблице симбола дефинише компајлирани модул. Процес омогућава имплементацију засебне компајлације (енг. *separate compilation*). Преведени модул се чува као независна датотека коју друге компајлационе јединице могу увести. Могућност увоза елиминише потребу за поновном синтаксном анализом и анализом изворног кода спољних компоненти [6]. Серијализација и увоз модула су приказани на слици 4.



Слика 4: Процес серијализације компајлираног модула и његовог увоза током компајлације.

Приликом избора механизма за перзистентност IR, основна подела обухвата текстуалне и бинарне формате [15]. Текстуални формати омогућавају читљивост и лако отклањање грешака. Међутим, текстуални форма-

ти уводе рачунарски трошак (енг. *parsing overhead*). Разлог је потреба за лексичком и синтаксном анализом при сваком читавању датотеке. Бинарни формати компактно записују структуре података у облик близак изворном меморијском распореду. Компактно записивање смањује заузеће простора на датотечном систему и убрзава читавање већ компајлираних модула.

Бинарни формати се деле на системе са строго дефинисаном шемом и самоописујуће формате без унапред одређене шеме (енг. *schemaless*). Системи са шемом (нпр. *Protocol Buffers* и *Apache Avro*) захтевају претходно моделовање података [15]. Пре покретања програма неопходно је и извршавање алата за генерисање изворног кода. Фиксна шема пружа максималну компактност јер бинарни запис не садржи називе поља. Са друге стране, крутост структуре уноси инжењерски трошак уколико се IR често мења. Самоописујући формати (нпр. *MessagePack*) чувају метаподатке о типовима заједно са самим вредностима. Складиштење пратећих ознака омогућава еволуцију објектног модела HIR-а у раним фазама развоја. Избор самоописујућег решења спаја флексибилност текстуалних записа са перформансама бинарног кодирања.

Формат *MessagePack* представља бинарну алтернативу JSON запису која задржава његову флексибилност, али значајно смањује просторну комплексност [14]. Ефикасност се постиже коришћењем типа префикса (енг. *type prefix*), где један бајт истовремено дефинише и тип податка и његову дужину. Тип префикса елиминише потребу за знаковима раздвајања. Карактеристике *MessagePack* формата омогућава мапирање релационих графова у компактне структуре бајтова. Читавање спољних модула своди се на секвенцијално читање меморије.

2.5 Генерисање података

Генерисање тест података на основу декларативних спецификација заснива се на стохастичком моделовању и рачунарској симулацији [16]. Статичка ограничења из доменског језика преводе се у излазне записе применом алгоритама случајног избора. Алгоритми случајног избора симулирају обрасце података у софтверским системима.

2.5.1 Псеудо-случајни генератори

Симулација у рачунарским системима ослања се на псеудо-случајне генераторе бројева (енг. *Pseudo-Random Number Generator*, PRNG) [17]. Рад алгорита отпочиње прихватањем почетне вредности под називом семе (енг. *seed*). Генератор користи семе за креирање низа бројева кроз сукцесивне математичке трансформације [18]. Математичке трансформације се најчешће заснивају на модуларној аритметици или манипулацији битовима [18]. Резултујући низ је детерминистички условљен почетном вредношћу. Излазни подаци пролазе статистичке тестове независности [17]. Примењени модел симулира својства случајности [17]. Избор PRNG алгорита утиче на укупне перформансе система. Избор алгорита одређује брзину извршавања рачунарских симулације и статистички квалитет генерисаних тест примера.

2.5.2 Униформна расподела

Униформна расподела је статистичка расподела у којој сви исходи унутар дефинисаног интервала $[a, b]$ имају једнаку вероватноћу избора [16]. Функција густине расподеле за непрекидни случај дефинисана је као:

$$\mathcal{U}(x) = \frac{1}{b - a}$$

за

$$a \leq x \leq b$$

У контексту система за генерисање података, када корисник дефинише нумерички или текстуални опсег (нпр. распон година или интервал датума) без навођења додатних метаподатака о структури, систем примењује дискретну или непрекидну униформну расподелу [18]. Униформна расподела додељује једнаку вероватноћу свим вредностима унутар дефинисаног опсега.

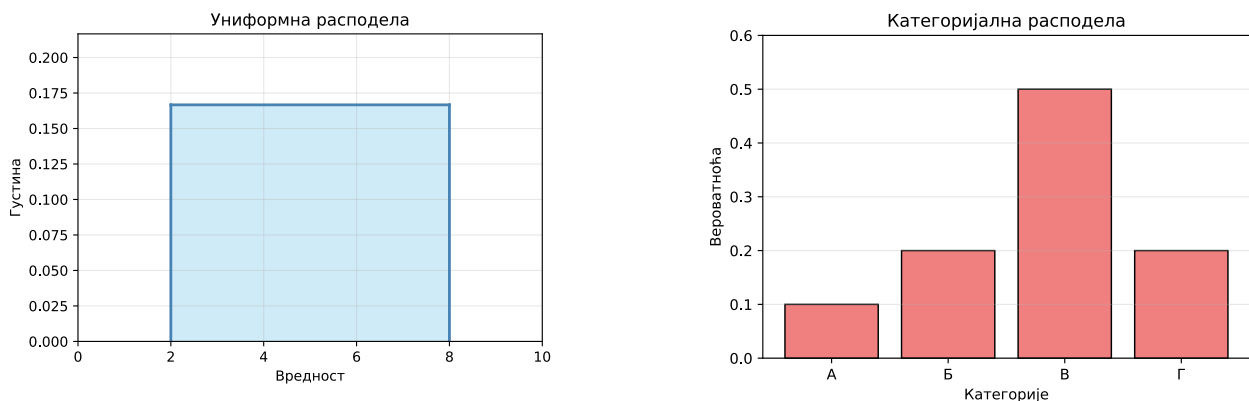
2.5.3 Тежински избор

Тежински избор (енг. *weighted selection*) је рачунарска имплементација категоријалне расподеле. Сваки потенцијални исход x_i поседује дефинисан фактор тежине w_i који одређује његову важност [16]. Вероватноћа избора појединачног исхода p_i израчунава се нормализацијом његове тежине у односу на суму свих тежина у скупу:

$$p_i = \frac{w_i}{\sum_{j=1}^n w_j}$$

Алгоритамски, тежински избор се реализује конструисањем низа кумулативних сума [18]. Систем генерише случајан број U из униформне расподеле у интервалу $[0, \sum w_j)$, а затим претрагом проналази индекс сегмента коме тај број припада. Временска комплексност алгоритма зависи од одабраног алгоритма претраживања. Секвенцијално претраживање низа уноси линеарну временску сложеност реда $\mathcal{O}(N)$, где вредност N представља укупан број категорија. Примена бинарне претраге оптимизује алгоритам и редукује комплексност на $\mathcal{O}(\log N)$. Тежински избор обезбеђује моделовање система у којима поједини ентитети имају унапред одређену, већу учесталост појављивања.

Теоријски приказ функције густине униформне расподеле и расподеле вероватноћа категоријалне расподеле дат је на слици 5. Са леве стране приказана је константна густина униформне расподеле на интервалу $[a, b]$, док су са десне стране приказане вероватноће избора појединачних категорија, одређене њиховим тежинама.



Слика 5: Теоријски приказ униформне (лево) и категоријалне расподеле (десно).

2.5.4 Репродуктивност

Репродуктивност представља особину система за стохастичко генерисање података која подразумева да идентична улазна спецификација доводи до конзистентних излазних резултата [16]. Фиксирање почетне вредности омогућава репликацију интерног стања генератора. Поновљивост обезбеђује детерминистичку репродукцију грешака откривених током извршавања тестова. Стабилност излазних структура оптимизује радне процесе унутар система за аутоматизовану интеграцију. Једнообразност података олакшава размену специфичних тест сценарија међу члановима развојног тима.

2.5.5 Референцијални интегритет и графови зависности

Генерисање сетова података који садрже међусобне везе захтева очување референцијалног интегритета [19]. У контексту синтетизације података, референцијални интегритет је структурно ограничење у коме вредност једног атрибута (страног кључа) мора постојати унутар скупа вредности референцираног ентитета [19]. Ограничење референцијалног интегритета онемогућава независно генерисање вредности и уводи релациону међузависност унутар система.

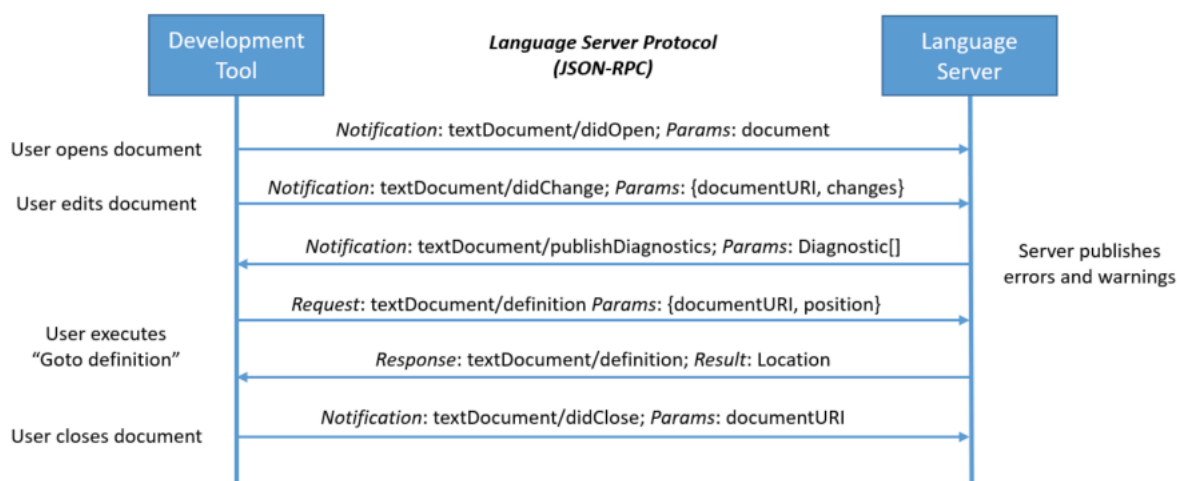
Међусобне условљености се моделују помоћу усмерених графова зависности (енг. *dependency graph*) [7]. Чворови графа представљају ентитете, док усмерене гране означавају постојање референцијалних

веза између њих. Проналажење валидног редоследа извршавања генерације захтева примену алгоритма за тополошко сортирање над конструисаним графом [7]. Тополошко сортирање поставља чворове графа у линеаран низ у коме сваки референцирани ентитет претходи зависном ентитету. Употреба тополошког сортирања успоставља редослед којим се сви примарни објекти креирају у меморији пре отпочињања обраде зависних поља.

2.6 Протокол језичких сервера

LSP је стандардни протокол комуникације између развојних окружења и језичких сервера [3]. Стандард раздваја језички неутралан уредник текста од језичког сервера [5]. Језички сервер извршава статичку и семантичку анализу без познавања специфичности развојног окружења [5]. Развојно окружење управља корисничким интерфејсом без познавања дубље семантике програмског језика [5].

Едитор и језички сервер комуницирају путем *JavaScript Object Notation Remote Procedure Call* (JSON-RPC) протокола [3]. Током уређивања, *едитор* шаље садржај документа и корисничке захтеве, док језички сервер враћа резултате анализе и податке потребне за функционалности *едитора* [4]. Слика 6 приказује ток комуникације.



Слика 6: Комуникација између *едитора* и језичког сервера током уређивања документа [3].

2.6.1 Проблем развоја језичке подршке

Развој подршке за развојна окружења захтевао је засебну имплементацију за сваки програмски језик [5]. Приступ доводи до укупног броја имплементација који одговара производу броја језика и броја развојних окружења [2]. Последице децентрализоване архитектуре обухватају високе трошкове развоја и спорију испоруку софтверских алата [4]. Различита развојна окружења често показују неконзистентно понашање приликом анализе истог изворног кода [4]. Комплексност одржавања онемогућава развојним тимовима пружање квалитетне подршке за више уредника текста [5].

2.6.2 Архитектура протокола језичког сервера

Архитектура користи клијент-сервер модел за раздвајање логике уредника текста од семантике језика [3]. Развојно окружење преузима улогу клијента који управља корисничким интерфејсом [5]. Језички сервер ради као независан процес задужен за статичку анализу кода [2]. Комуникација се ослања на стандардизоване поруке унутар JSON-RPC протокола [3]. Протокол дефинише одређен животни циклус сесије кроз захтеве за иницијализацију и гашење сервера [3]. Размена података обухвата асинхронно слање обавештења о изменама унутар отворених датотека [3]. Сервер извршава анализу у позадини и самостално шаље дијагностичке поруке клијенту [3].

2.6.3 Функционалности протокола језичког сервера

Протокол стандардизује функционалности за подршку током кодирања [3]. Основна могућност обухвата аутоматско допуњавање кода на основу тренутног контекста [3]. Систем омогућава прелазак на место дефиниције симбола у изворном коду [3]. Функционалност претраге референци проналази сва појављивања изабраног идентификатора у пројекту [3]. Приказ информација кроз лебдећи прозор пружа увид у типове података и документацију симбола [3]. Генерисање дијагностичких порука приказује синтаксне и семантичке грешке у реалном времену [2]. Подршка за рефакторисање укључује сигурно аутоматизовано преименовање симбола кроз све датотеке [3].

2.6.4 Конкурентност и механизам отказивања задатака

Унутар LSP-а, перформансе и одзивност корисничког интерфејса зависе од асинхроне обраде захтева [3]. Корисник непрекидно мења изворни код брзим уносом карактера, процеси семантичке анализе могу постати застарели пре него што се изврше до краја. Да би се спречило непотребно заузеће процесорских ресурса, LSP уводи механизам отказивања задатака путем токена за отказивање (енг. *cancellation token*) [3]. Клијент шаље обавештење о отказивању за сваки захтев чији је резултат постао сувишан услед новог стања у *едитору*. Језички сервер периодично проверава статус токена током дуготрајних операција, прекида тренутно извршавање у случају активације и ослобађа ресурсе за наредну итерацију анализе.

Пројектовање и имплементација алата за генерисање података заснованих на наменским језицима захтевају дефинисање тренутног технолошког стања и сродних софтверских архитектура. Развој решења обухвата три области: инжењерство програмских језика, системе за дефинисање формалних шема података и платформе за стохастичку синтетизацију тест примера.

Унутар поглавље се приказује преглед постојећих алата унутар наведених категорија. Анализа идентификује предности и ограничења решења, са фокусом на начине разрешавања референцијалног интегритета, експресивности синтаксе и интеграције са *едиторима*. Упоредни приказ служи као основа евалуацију која оправдава развој новог доменског језика и пратеће компајлерске архитектуре.

3.1 Језици специфични за домен

Развој DSL-а најчешће се не изводи изградњом компајлера од почетног нивоа, већ применом специјализованих мета-језичких алата и развојних окружења који се називају језичке радионице (енг. *language workbench*). Системи омогућавају декларативно дефинисање граматике, семантичких правила и пратећих компоненти за уређивање кода [6]. Анализирани алати показују значајне разлике у погледу комплексности архитектуре и интеграције са екстерним системима.

3.1.1 *textX*

TextX је мета-језик имплементиран у програмском језику *Python*. Намењен је за изградњу DSL-а [20]. Систем се заснива на синтаксном анализатору *Arpeggio*. *Arpeggio* користи концепт граматике анализе синтаксних израза (енг. *Parsing Expression Grammar*) [21]. На основу текстуалне граматике, *textX* конструише мета-модел у меморији и извршава валидацију улазних датотека.

Предности алата су брзо прототипизовање и интеграција са *Python* екосистемом. Корисник дефинише језичка правила кроз синтаксу сличну проширеној *Backus-Naur-овој* форми (листинг 1). Систем мапира препознате синтаксне структуре у динамичке објекте. Мапирање смањује когнитивно оптерећење инжењера, јер развој не захтева ручно писање класа за AST.

```
Model: entities+=Entity;
Entity: 'entity' name=ID '{' properties+=Property '}';
Property: name=ID ':' type=ID;
```

Листинг 1: Дефиниција граматике у алату *textX*

Архитектура алата *textX* показује ограничења у домену стохастичке синтезе података и аутоматизације IDE подршке. Систем не поседује изворне семантичке операторе за дефинисање тежинских вероватноћа унутар саме граматике. Недостатак тежинских вероватноћа онемогућава управљање расподелом случајних вредности на нивоу језика. Алгоритам за разрешавање референци унутар алата *textX* ограничен је на једноставно повезивање објеката. Комплексни граф зависности и валидација референцијалног интегритета морају се имплементирати ручно, кроз *Python* код након фазе синтаксне анализе. Изворна интеграција са развојним окружењима кроз LSP није део језгра система, већ захтева развој додатних наменских библиотека.

3.1.2 *JetBrains MPS*

JetBrains MPS је језичка радионица заснована на концепту пројекционог уређивања кода [22]. Систем не користи синтаксни анализатор за претварање текстуалног записа у AST. Корисник модификује структуру стабла у меморији. Измена кода се извршава кроз наменске визуелне пројекције унутар окружења.

Предност *JetBrains MPS* је подршка за комбиновање различитих нотација унутар истог модела [23]. Код се може приказати у облику текста, табела, математичких формула или графичких дијаграма. Непостојање фазе синтаксне анализе елиминише појаву синтаксних двосмислености током дефинисања језика. Окружење омогућава поновну употребљивост развијених језичких компоненти.

Архитектура *JetBrains MPS* показује ограничења у погледу интеграције и намене за синтезу података. Систем захтева коришћење монолитног развојног окружења. Захтев онемогућава једноставну интеграцију са спољним текстуалним уредницима путем LSP протокола. Пројекциони приступ уводи когнитивно оптерећење при уносу кода због одсутности текстуалне синтаксе [23]. Систем не поседује изворне семантичке операторе за дефинисање стохастичких вероватноћа. Разрешавање комплексних релационих зависности између ентитета захтева ручно писање трансформационих генератора у језику *BaseLanguage* [22].

3.2 Системи за дефинисање шема

Системи за дефинисање структура и шема података примарно су намењени валидацији, серијализацији и размени информација унутар дистрибуираних софтверских архитектура [15]. Иако изворна намена није стохастичко генерисање података, технологије поседују формалне описе ентитета и релација. Користе се као полазна основа за аутоматизовано креирање тест сценарија, при чему се уочавају ограничења услед недостатка изворних стохастичких оператора унутар мета-модела.

3.2.1 JSON Schema

JSON Schema је формат за декларативно дефинисање и валидацију структура унутар JSON записа [24]. Систем омогућава постављање структуралних ограничења над подацима унутар дистрибуираних софтверских архитектура [15] (листинг 2). Технологија служи за верификацију исправности размене информација између независних апликација.

JSON Schema подржава поновну употребљивост компоненти помоћу механизма синтаксних показивача [24]. Употреба оператора *\$ref* омогућава експлицитно повезивање локалних и удаљених објеката унутар дефинисане шеме. Провера усклађености записа изводи се аутоматски преко софтверских библиотека доступних у GPL.

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "Course",
  "type": "object",
  "properties": {
    "course_id": { "type": "integer" },
    "name": { "type": "string", "pattern": "^Kurs-[0-9]{3}$" },
    "department": { "type": "string", "enum": ["ComputerScience", "Mathematics"] }
  },
  "required": ["course_id", "name", "department"]
}
```

Листинг 2: Пример *JSON Schema*

Примена формата у домену стохастичке синтезе података показује недостатке. Мета-модел не поседује изворне семантичке операторе за дефинисање тежинских вероватноћа избора вредности [24]. Аутоматизовано креирање тест сценарија на основу *JSON Schema* захтева имплементацију нестандартних екстензија изван званичне спецификације.

3.2.2 Protocol Buffers

Protocol Buffers је језик за дефинисање интерфејса (енг. *Interface description language*) и формат за бинарну серијализацију података [25]. Технологија омогућава размену информација унутар микросервисних архитектура [15]. Спецификација модела се дефинише декларативно унутар изворних текстуалних датотека са екстензијом *.proto* (листинг 3).

Предности *Protocol Buffers* су перформансе и компактност генерисаног бинарног записа [15]. Наменски компајлер *protoc* преводи дефинисане поруке у структуре података за различите програмске језике [25]. Формат подржава унапредиву компатибилност шема кроз систем нумеричких ознака поља.

```
syntax = "proto3";

message Korisnik {
    int32 korisnik_id = 1;
    string ime = 2;
}
```

Листинг 3: Шема поруке унутар *proto* датотеке

Архитектура система *Protocol Buffers* показује ограничења у контексту генерисања тест података. Мета-модел нема изворне семантичке операторе за конфигурисање стохастичких вероватноћа унутар шеме [25]. Систем не поседује механизме за аутоматско препознавање и очување референцијалног интегритета између засебних порука. Разрешавање комплексних релација између ентитета захтева имплементацију додатних алгоритама на нивоу корисничке апликације.

3.3 Системи за генерисање тест података

Системи за генерисање тест података чине категорију софтверских решења за аутоматизацију тестирања. Развојни фокус је синтеза реалистичних вредности унутар база података и корисничких интерфејса. Употреба наменских генератора смањује ручни напор приликом креирања сложених тест сценарија.

3.3.1 *Faker*

Faker је програмска библиотека доступну у већини GPL, намењену за генерисање псеудослучајних података [26]. Интеграција у софтверске пројекте изводи се кроз изворни код тест оквира (листинг 4).

Предност *Faker* библиотеке је флексибилност унутар развојног окружења. Корисник комбинује генерисане вредности са пословном логиком апликације. Локализовани провајдери омогућавају прилагођавање излазних стрингова језичким и културолошким форматима.

```
from faker import Faker
generator = Faker()

print(generator.name())
print(generator.email())
```

Листинг 4: Имплементација псеудослучајног генератора у језику *Python*

Архитектура библиотеке *Faker* уводи императивни трошак при моделирању релационих структура [26]. Развој захтева ручну конструкцију меморијских бафера за чување примарних кључева ради очувања референцијалног интегритета. Систем захтева експлицитно дефинисање редоследа извршавања петљи за сваки повезани ентитет. Алат не поседује изворне семантичке операторе за декларативно конфигурисање стохастичких тежина на нивоу модела података.

3.3.2 *Mockaroo*

Mockaroo је веб-платформа заснована на графичком корисничком интерфејсу (енг. *Graphical User Interface*, GUI) за генерисање реалистичних тест података (слика 7) [27]. Систем омогућава конфигурисање излазних датотека у форматима као што су CSV, JSON и SQL [27]. Платформа поседује уграђену базу генератора за различите индустријске и пословне домене [27].

Предност платформе је интуитивно дефинисање међусобних релација између ентитета кроз визуелне компоненте [27]. Систем подржава механизме за очување референцијалног интегритета помоћу унакрсног повезивања колона [27]. Корисници примењују наменске формуле за увођење стохастичких тежина и вероватноћа при синтези вредности [27].

Field Name

Type

Options

id

Row Number

blank:

0 %

Σ

×

first_name

First Name

blank:

0 %

Σ

×

last_name

Last Name

blank:

0 %

Σ

×

email

Email Address

blank:

0 %

Σ

×

gender

Gender

blank:

0 %

Σ

×

ip_address

IP Address v4

blank:

0 %

Σ

×

+ ADD ANOTHER FIELD

GENERATE FIELDS USING AI...

Rows:

1000

Format:

CSV

Line Ending:

Unix (LF)

Include:

☒ header

☐ BOM

Слика 7: GUI платформе *Mockaroo* за конфигурацију генератора података

Архитектура алата *Mockaroo* показује ограничења у погледу интеграције и локалне контроле. Алат функционише као затворена инфраструктура, што онемогућава локално извршавање унутар изолованих мрежа [27]. Одсуство текстуалне синтаксе отежава верзионисање и праћење измена модела кроз системе за контролу верзија [6]. Бесплатна верзија платформе ограничава максималан број генерисаних редова по датотеци [27].

3.4 Закључна разматрања и упоредни приказ

Евалуација технологија открива компромис између изражајности мета-модела и когнитивног оптерећења развоја [6]. Текстуални формати омогућавају интеграцију са системима за контролу верзија кода [2]. Графичке платформе убрзавају визуелну конфигурацију релација уз ограничење локалног извршавања софтвера [6]. Бинарни формати оптимизују мрежни пренос података уз губитак читљивости изворног записа [15]. Табела 2 садржи таксономију карактеристика, предности и ограничења свих анализираних решења.

Табела 2: Поређење предложеног решења са постојећим алатима

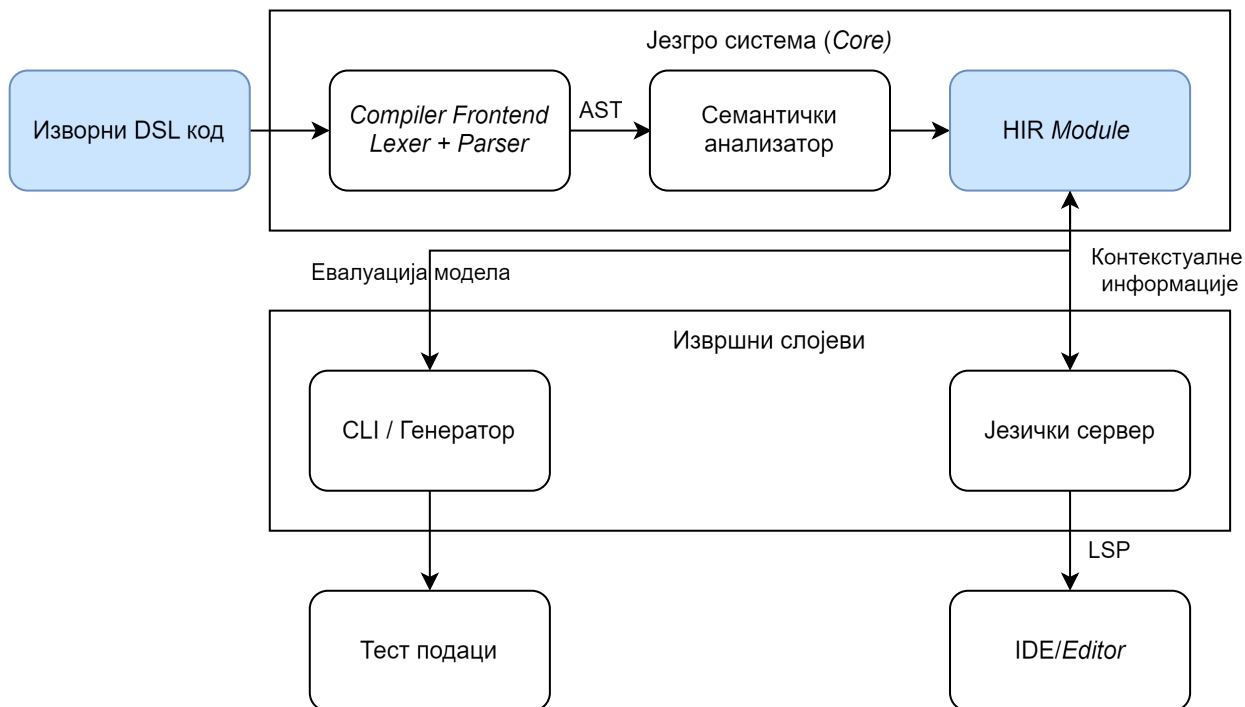
Алат / Систем	Приступ моделовању	Референцијални интегритет	Стохастички оператори	Изворни LSP
<i>textX</i>	Текстуални DSL	Делимично	Не	Не
<i>JetBrains MPS</i>	Пројекциони DSL	Да	Не	Не
<i>JSON Schema</i>	Шема (JSON)	Да	Не	Делимично
<i>Protocol Buffers</i>	Шема (бинарна)	Не	Не	Не
<i>Faker</i>	Програмски API	Ручно	Не	Не примењује се
<i>Mockaroo</i>	GUI	Да	Да	Не примењује се
Предложено решење	Текстуални DSL	Аутоматски (граф)	Да	Да

Циљ софтверског решења је развој DSL-а који омогућава генерисање тест података. Систем као улаз прима текст DSL датотеке и захтеве које *едитор* шаље путем LSP протокола. Излаз система чине текстуалне датотеке са изгенерисаним тест подацима и одговори формирану у складу са LSP спецификацијом.

Ово поглавље описује целокупан процес дизајна система. Архитектура система (поглавље 4.1) приказује структуру софтвера и међусобне везе између компоненти. Наредни сегмент, посвећен DSL-у (поглавље 4.2), дефинише конкретну синтаксу, граматичка правила и припадајуће структуралне елементе. Семантичка анализа (поглавље 4.3) обухвата начин имплементације правила контекстуалне исправности и процес валидације изворног кода. За премошћавање јаза између текстуалног записа и саме генерације тест података задужен је HIR (поглавље 4.4), који уводи специфичан апстрактни модел. Рашчлањивање унутрашње структуре софтвера на независне компоненте уз дефинисање њихове организације изложено је у оквиру дизајна модула (поглавље 4.5). Дизајн генератора података (поглавље 4.6) описује методе за извршавање стохастичког узорковања и одржавање конзистентности референци. Напоследку, део о језичком серверу (поглавље 4.7) описује архитектуру подсистема за прихватање LSP захтева, управљање стањем отворених докумената и формирање одговора за развојно окружење.

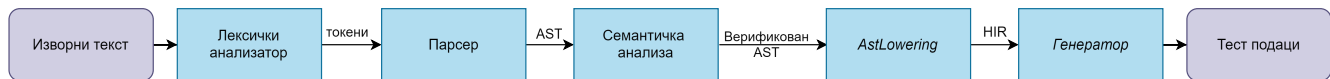
4.1 Архитектура система

Архитектура система пројектована је по принципу дељеног језгра. Логика лексичке, синтаксне и семантичке анализе користи у различитим модулима извршавања. Глобална структура софтвера и међусобне везе између главних компоненти приказане су на слици 8.



Слика 8: Дијаграм архитектуре система

Процес превођења изворне спецификације у финални извештај одвија се кроз неколико узастопних фаза. Ток података и постепена трансформација модела кроз ланац обраде (енг. *pipeline*) у компајлеру илустровани су на слици 9.



Слика 9: Ланац обраде у компајлеру

4.2 Дизајн језика специфичног за домен

Дизајн DSL-а се заснива на декларативном принципу. Корисник описује структуру и ограничења података. Поступак генерисања је потпуно апстрахован. Систем је пројектован као статички типизиран језик. Статичка типизација омогућава рано откривање грешака у спецификацији путем језичког сервера [1], [6].

4.2.1 Директиве и конфигурација контекста

Синтаксне директиве почињу симболом `@`. Директиве служе за глобалну конфигурацију окружења и управљање модулима. Ови конструкти дефинишу метаподатке преводиоца. Пример глобалних директива и увоза модула приказан је на листингу 5.

```

@import skalarne_definicije;

@output json { pretty = true; indent = 4; }
@output_path "./izlaz/korisnici.json";

@generate Korisnik[100];

```

Листинг 5: Пример глобалних директива и увоза модула

Директива `@import` омогућава декомпозицију комплексне спецификације на мање модуле. Директива `@output` дефинише излазни формат JSON (енг. *JavaScript Object Notation*) и пратећу индентацију. Директива `@output_path` одређује циљну локацију датотеке на диску. Извршна директива `@generate` покреће генерисање над изабраним шаблоном.

4.2.2 Кориснички типови и синтаксна ограничења

Систем типова подржава проширење примитивних типова кроз увођење корисничких типова са ограничењима. Дефинисање скаларних типова са ограничењима илустровано је на листингу 6.

```

type GodinaStudija = int [range=1..=4];
type ProsecaOcena = float [range=6.0..=10.0];
type BrojIndeksa = string_pattern "2026/${#[4]}";

```

Листинг 6: Дефинисање скаларних типова са ограничењима

Синтаксна конструкција `range` ограничава домен дозвољених нумеричких вредности. Генератор примењује кориснички дефинисану униформну расподелу вредности унутар задатог интервала. Спецификација `string_pattern` користи текстуалне макрое за форматирање псеудо-насумичних низова карактера.

4.2.3 Колекцијски типови и вишедимензионалне структуре

Систем типова подржава дефинисање колекција кроз типске листе. Листе омогућавају груписање више елемената истог базног типа. Број генерисаних елемената контролише се синтаксним ограничењем `count`. `Count` дефинише минималан и максималан број чланова колекције. Дефинисање колекцијских типова и уложених структура приказано је на листингу 7.

```

type Grade = int [range=6..10];
type IntList = [int][count=1..5];
type IntMatrix = [IntList][count=1..5];

template Student {
    grades = [Grade][count=1..5];
    tensor = [IntMatrix][count=1..5];
}

```

Листинг 7: Дефинисање колекцијских типова и вишедимензионалних структура

Конструкт листе прихвата примитивне и кориснички дефинисане скаларне типове. Типски систем дозвољава вишеструко улагање листа. Улагање омогућава креирање сложених вишедимензионалних структура попут матрица и тензора. Генератор стохастички одређује коначну дужину сваке листе унутар дефинисаног интервала приликом инстанцирања шаблона.

4.2.4 Пробабилистичке енумерације

Енумерације дефинишу коначан скуп вредности и припадајуће тежинске факторе расподеле. Тежински фактори се експлицитно наводе помоћу оператора `=>`. Пример енумерације са факторима тежине расподеле приказан је на листингу 8.

```

enum StatusStudenta {
    Budzet          => 7;
    Samofinansiranje => 3;
}

```

Листинг 8: Енумерација са факторима тежине расподеле

Генератор користи дефинисане тежине за стохастичко узорковање вредности. Вероватноћа избора ставке буџет износи тачно 70% док за самофинансирање износи 30%. Пробабилистичке енумерације омогућавају подешавање статистичког профила излазних података.

4.2.5 Структурна композиција и наслеђивање

Моделовање сложених објеката извршено је кроз концепте структура (енг. *struct*) и шаблона (енг. *template*). Структура дефинише чист логички контејнер за композицију поља. Шаблон представља активну семантичку јединицу над којом се извршава генерација. Наслеђивање поља дефинише се оператором `:`. Структурна композиција и механизам наслеђивања приказани су на листингу 9.

```

struct Lokacija {
    grad    = string;
    drzava  = string;
}

template Osoba {
    ime     = string;
    adresa  = Lokacija;
}

template Zaposleni : Osoba {
    plata   = int [range=50000..150000];
}

```

Листинг 9: Структурна композиција и механизам наслеђивања

Шаблон *Zaposleni* аутоматски наслеђује сва поља родитељског шаблона *Osoba*. Наслеђивање обухвата и уложено поље *adresa*. Наслеђивање смањује редундантност приликом спецификације сложених домена. Композиција омогућава хијерархијску организацију података унутар резултујућег записа.

4.2.6 Очување релационог интегритета

Очување логичких веза између генерисаних ентитета реализовано је увођењем кључне речи *ref*. Референцирање вредности ради очувања интегритета илустровано је на листингу 10.

```
template Smer {
    sifra = string_pattern "${A[3]}";
}

template Student {
    id      = int;
    smer_kod = ref Smer.sifra;
}
```

Листинг 10: Референцирање вредности ради очувања интегритета

Кључна реч *ref* сигнализира преводиоцу да вредност поља мора бити преузета из скупа већ изгенерисаних вредности циљног шаблона. Очување релационог интегритета симулира спољне кључеве из релационих база података. Спољни кључеви спречавају појаву неконзистентних података у финалном извештају.

4.3 Дизајн семантичке анализе

Семантичка анализа представља средишњи део аналитичког слоја унутар архитектуре система. Синтаксни анализатор прослеђује изграђен AST семантичком слоју. Семантички анализатор обавља три фазе провере. Излаз ове компоненте обухвата попуњену табелу симбола и скуп дијагностичких порука.

Изградња табеле симбола чини почетну фазу семантичке анализе. Компонента за изградњу табеле симбола пролази кроз структуру стабла помоћу дизајнерског шаблона посетилац [12]. Модул региструје све дефинисане шаблоне, структуре, енумерације и корисничке типове. Компонента прати контекст извршавања кроз улазак у опсеге видљивости и излазак из њих. Систем детектује невалидне контексте попут дефинисања поља изван дозвољених структура. Анализатор проналази дуплиране идентификаторе унутар истог опсега.

Провера типова представља другу фазу семантичке анализе. Модул за проверу типова анализира исправност израза и додељених вредности. Компонента утврђује конкретан тип за сваки конструкт у спецификацији. Систем захтева да тежински фактори унутар енумерација буду искључиво целобројне вредности. Компонента такође контролише број генерисаних ентитета у извршним директивама. Наведени број мора одговарати целобројном типу података. Свако одступање од ових правила резултује генерисањем семантичке грешке о неслагању типова.

Провера референци је завршна фаза семантичке анализе. Компонента контролише постојање свих коришћених идентификатора. Компонента потврђује исправност кључне речи *ref* приликом повезивања поља између различитих шаблона, претражује локалну табелу симбола и табеле симбола из увезених модула. Провервач референци детектује непостојеће родитељске шаблоне у процесу наслеђивања. Фаза очувања интегритета спречава позивање непостојећих поља унутар структура.

4.4 Дизајн високонивојске међурепрезентације

Процес преводијења из AST-а у HIR изводи компонента *AstLowering* [7]. *AstLowering* компонента уклања синтаксно-декоративне елементе и трансформише структуре у облике погодне за генерацију података и рад језичког сервера. Процес трансформације користи изоловани контекст спуштања. Контекст спуштања омогућава једнопролазну регистрацију свих дефиниција. Регистрованим дефиницијама се валидира постојање пре доделе јединствених идентификатора.

HIR обухвата интернирање стрингова, адресирање ентитета, спуштање израза и издвајање ограничења. Базен стрингова преводи текстуалне идентификаторе у нумеричке кључеве [7]. Поређење стрингова своди се на операције над целим бројевима. Архитектура система ослања се на концепт униформног глобалног адресирања. Ентитети се идентификују композитним кључем. Композитни кључ спаја идентификатор

модула и локални идентификатор ставке унутар тог модула. Поља шаблона и структура користе засебне нумеричке идентификаторе. Семантички граф програма постаје независан од оригиналног текстуалног записа. Нумеричка репрезентација омогућава претрагу графа и бинарну серијализацију.

Крајњи резултат процеса спуштања је структура *Module* (листинг 11). *Module* служи као централни контејнер који обједињује све релевантне аспекте једне компајлиране датотеке.

```
struct Module {
    metadata: ModuleMetadata,
    string_pool: StringPool,
    imports: Vec<StringId>,
    directives: Vec<Directive>,
    items: Vec<Item>,
    source_map: SourceMap,
}
```

Листинг 11: Концептуални модел HIR модула

Module структура концептуално повезује метаподатке, базен стрингова, увозне зависности, извршне директиве, компајлиране ставке и мапу изворног кода. Модул се посматра као јединствена, самодовољна целина, погодна за пренос између различитих фаза компајлера.

4.5 Дизајн модула

Дизајн модуларног система инспирисан је архитектуром изолованих метаподатака компајлера *rustc* [11]. Архитектура омогућава организацију дефиниција кроз више датотека. Систем подржава симултани рад језичког сервера са више табела симбола. Систем почива на четири дизајнерске компоненте:

- међумодуларно референцирање (поглавље 4.5.1),
- релокација модула и разрешавање зависности (поглавље 4.5.2),
- перзистентност и серијализација (поглавље 4.5.3) и
- реконструкција стања за језички сервер (поглавље 4.5.4).

4.5.1 Међумодуларно референцирање

Повезивање симбола између различитих модула користи механизам композитних кључева. Дизајн елиминира потребу за засебним структурама података при раду са локалним и спољашњим симболима. Апстрахује се физичка локација дефиниције ентитета. Разрешавање локалних ставки захтева поређење идентификатора модула са тренутним контекстом превођења. Непоклапање идентификатора преусмерава претрагу на мапу увезених зависности. Систем задржава конзистентну репрезентацију графа независно од укупног броја улазних датотека.

4.5.2 Релокација модула и разрешавање зависности

Независно превођење датотека захтева раздвајање фазе превођења од фазе повезивања модула у меморији. Нумерички идентификатори унутар учитаног модула захтевају усклађивање са адресним простором тренутног процеса преводилаца. Систем покреће фазу релокације након читавања зависности. Алгоритам за релокацију рекурзивно обилази комплетно стабло учитане датотеке. Централни разрешавач додељује нове вредности идентификаторима. Старе вредности идентификатора модула транслирају се у додељене нове вредности. Примењени механизам опонаша концепт динамичког повезивања. Повезивање у меморији одржава конзистентност модела при раду са великим бројем међузависних датотека.

4.5.3 Перзистентност стања

Да би се избегло поновна синтаксна и семантичка анализа зависних датотека, дизајн предвиђа чување анализираног HIR модела на диску. Као формат серијализације изабран је *MessagePack* [14]. *MessagePack* формат обезбеђује компактан бинарни запис и брзу десеријализацију [14]. Централна структура *Module* пројектована је тако да буде потпуно самодовољна приликом уписа и читавања са диска.

Употреба типизираних бинарних формата са датотекама шеме уводи развојни трошак при честим изменама HIR модела [15]. Свака промена унутар преводаца захтевала би ажурирање екстерне шеме. *MessagePack* формат подржава серијализацију самоописујућих бинарних структура без унапред дефинисане шеме. Употребом *MessagePack* формата је очувана флексибилност измене HIR модела уз задржавање перформанси читавања.

4.5.4 Реконструкција стања за језички сервер

За потребе рада језичког сервера, дизајн обухвата механизам трансформације статичког HIR модела назад у динамичку табелу симбола. Процес захтева итерацију кроз све компајлиране ставке унутар модула и поновно мапирање њихових својстава. Реконструкција омогућава језичком серверу да креира контекст изолованих датотека и изврши ефикасну валидацију међумодуларних веза [11].

4.6 Дизајн генератора података

Генератор података трансформише HIR у текстуални извештај. Синтезу покрећу извршне директиве за генерисање. Архитектура генератора обухвата:

- модул за евалуацију ограничења (поглавље 4.6.1),
- модул за стохастичко узорковање (поглавље 4.6.2) и
- модул за очување референцијалног интегритета (поглавље 4.6.3).

4.6.1 Модул за евалуацију ограничења

Модул за евалуацију ограничења анализира правила дефинисана над корисничким типовима. Вредности за нумеричке и текстуалне променљиве изолују се из HIR-а пре почетка синтезе. Анализатор мапира декларативна ограничења у математичке интервале и дефинисане скупове. Вредности унутар дефинисаних опсега генеришу се применом континуалне или дискретне униформне расподеле [17].

4.6.2 Модул за стохастичко узорковање

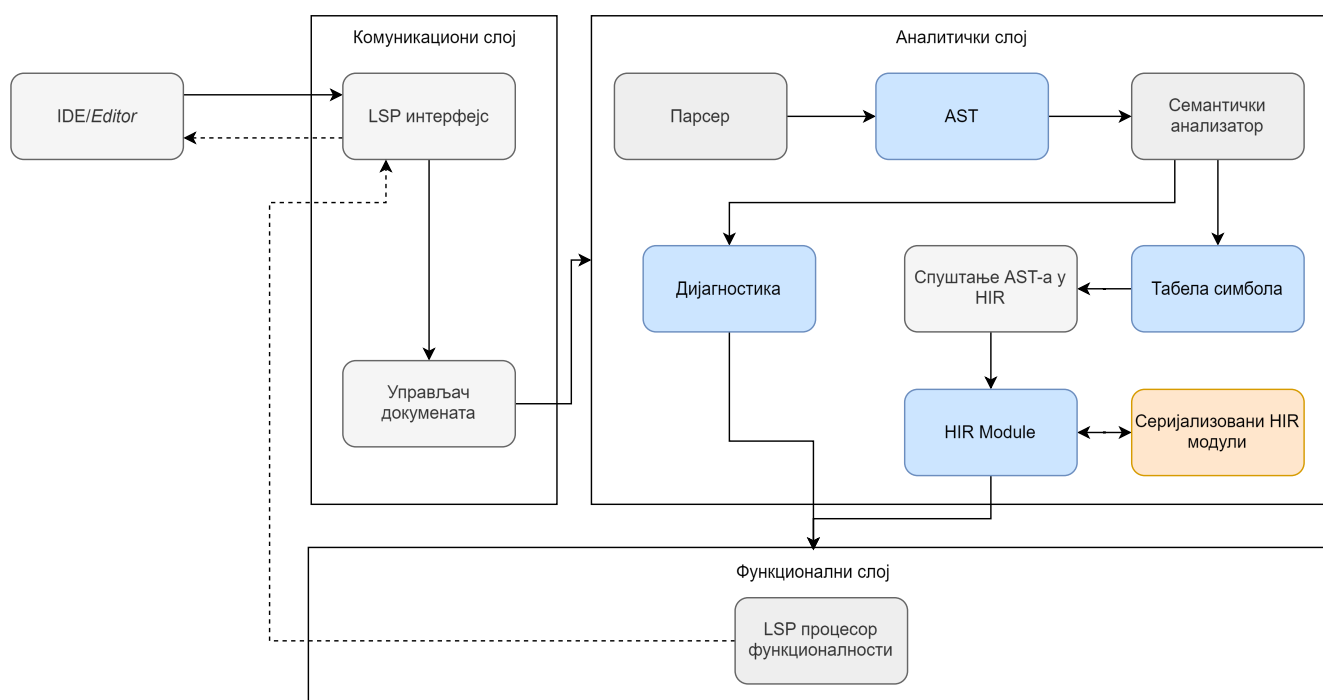
Модул за стохастичко узорковање управља синтезом случајних вредности и избором категорија. Израчунавање се заснива на PRNG [17]. Иницијализација генератора одређеном почетном вредношћу (енг. *seed*) обезбеђује детерминистички излаз и репродуктивност синтезе података [28]. Избор вредности из пробабилистичких енумерација изводи се алгоритмом за тежински избор [28]. Свакој варијанти додељује се пропорционални део јединичног интервала на основу нормираних тежинских фактора. Псеудо-случајна променљива одређује коначни избор према дефинисаним вероватноћама расподеле.

4.6.3 Модул за очување референцијалног интегритета

Модул за очување референцијалног интегритета спречава појаву неконзистентних вредности унутар извештаја [29]. Зависне везане вредности захтевају изградњу оријентисаног графа зависности [30]. Чворови графа представљају шаблоне генерације, док усмерене гране означавају смерове референцирања кроз израз *ref*. Тополошко сортирање графа одређује редослед извршавања операција синтезе [30]. Процедура гарантује изградњу независних ентитета пре покретања генерације зависних објеката. Финални упис преузима вредности из привременог бафера већ изгенерисаних кључева.

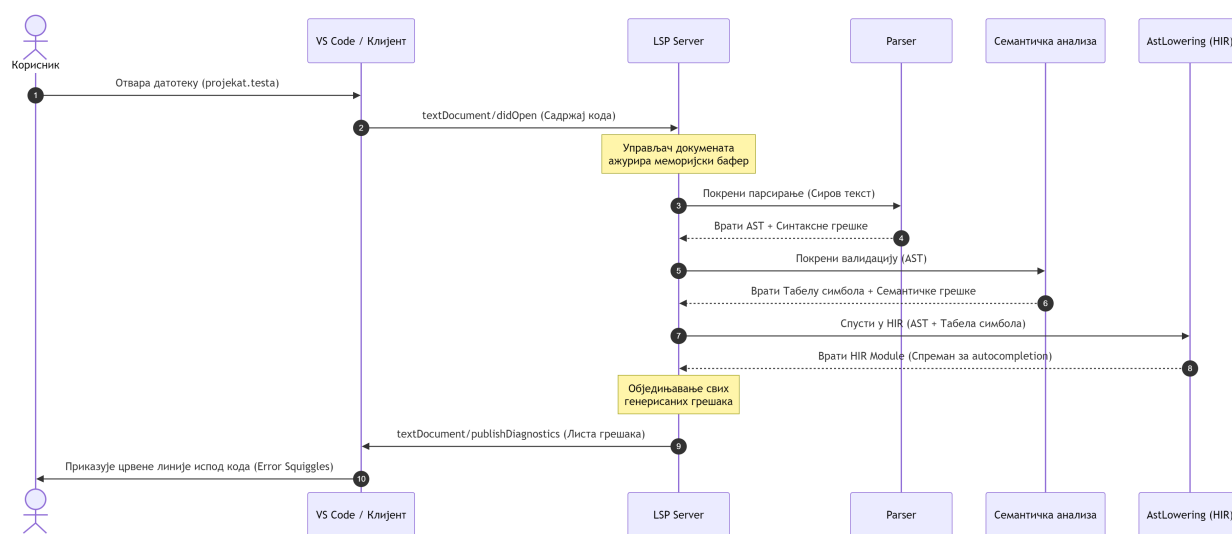
4.7 Дизајн језичког сервера

Језички сервер обухвата три логичка слоја: комуникациони слој (поглавље 4.7.1), аналитички слој (поглавље 4.7.2) и функционални слој (поглавље 4.7.3). Слика 10 приказује компоненте сваког слоја и ток обраде LSP захтева.



Слика 10: Архитектура LSP система и ток обраде LSP захтева.

LSP се заснива на асинхроној размени порука које су вођене спољним догађајима унутар развојног окружења. Временски редослед размене асинхроних порука и след активности унутар система приликом отварања датотеке приказани су на слици 11.



Слика 11: Секвенцијални дијаграм обраде догађаја при отварању датотеке

4.7.1 Комуникациони слој

Комуникациони слој представља улазну тачку система. Задужен је за размену порука између *едитора* и језичког сервера. Слој прима LSP захтеве, управља стањем отворених докумената и прослеђује поруке осталим компонентама система [3].

Улазну тачку комуникационог слоја представља LSP интерфејс. Компонента прихвата захтеве које *едитор* шаље и преводи их у интерне операције система [3]. Након обраде, компонента формира одговор и враћа га *едитору* [3].

Поред LSP интерфејса, комуникациони слој садржи управљач докумената (енг. *document manager*). Компонента прати измене над отвореним документима и одржава њихово стање за потребе даље обраде. Свака измена садржаја документа покреће поновну анализу и ажурирање дијагностичких порука.

Асинхрона природа JSON-RPC протокола узрокује ризик од појаве трке за ресурсе (енг. *race condition*) [3]. Учестали захтеви за измену документа пристижу брже него што систем завршава семантичку анализу претходног стања. Трка за ресурсе може довести до неконзистентности података унутар компајлера. За решавање проблема дизајниран је механизам за отказивање (енг. *cancellation token*). Механизам за отказивање прекида активну нит позадинске анализе уколико из уредника пристигне нови унос. Ресурси сервера се преусмеравају на обраду последњег стања кода.

4.7.2 Аналитички слој

Аналитички слој врши обраду DSL докумената и припрема податке потребне за LSP функционалности. Слој обухвата синтаксну анализу, семантичку анализу и изградњу интерних модела података. Резултат анализе представља HIR, која садржи семантички модел програма независан од синтаксног записа. Кроз серијализацију табеле симбола, HIR омогућава подршку за независне модуле. Језичком серверу HIR пружа могућност симултаног приступа већем броју међусобно повезаних табела симбола. HIR се користи као заједничка основа за LSP функционалности, генерацију тест података и серијализацију модула.

Обрада започиње синтаксном анализом. Синтаксни анализатор трансформише текст документа у AST [7]. У случају уочавања синтаксних неправилности, синтаксни анализатор примењује механизме опоравка од грешака како би наставио са обрадом остатка документа. Овакав приступ омогућава систему да идентификује и прикаже више независних грешака истовремено, уместо прекида обраде при првој грешци [7].

Над генерисаним AST-ом извршава се семантичка анализа. Компонента проверава типове, валидност референци и конзистентност конструкција дефинисаних DSL правилима. Током анализе, формира се локална табела симбола, која садржи дефиниције и везе између елемената програма [7]. Уочене неправилности се бележе кроз модел дијагностике. Модел садржи опис грешке, ниво озбиљности и локацију у документу [3].

Након успешно завршене семантичке анализе AST се трансформише у HIR. У HIR-у су разрешене референце, типови и односи између симбола, како из тренутног документа, тако и из претходно серијализованих увозних модула. Ово HIR чини погодним за трајно чување и поновно учитавање модуларних целина.

Развој синтаксног анализатора унутар језичког сервера захтева имплементацију опоравка од грешака (енг. *error recovery*). Пратов алгоритам прекида извршавање при првој синтаксној неправилности [13]. Прекидање рада није прихватљиво за рад у реалном времену где корисник пише непотпун код. Проблем је решен увођењем токена за синхронизацију унутар главне петље синтаксног анализатора. Систем у случају грешке изолује неисправан израз и конструише делимично стабло. Синтаксни анализатор наставља са рашчлањивањем остатка документа.

4.7.3 Функционални слој

Функционални слој имплементира кориснички видљиве операције трансформацијом интерних модела у одговоре дефинисане LSP спецификацијом. Слој обухвата компоненте које омогућавају:

- аутоматско допуњавање кода на основу тренутног контекста и симбола доступних у локалној табели симбола и унутар серијализованих HIR модела увезених модула,
- дијагностику грешака, коришћењем резултата семантичке анализе за формирање порука о неправилностима,
- навигацију кроз код која кориснику обезбеђује прелазак на дефиниције симбола кроз глобални опсег дефинисан учитаним табелама симбола и
- приказ додатних информација (енг. *hover*), пружањем контекстуалних описа интеракције са кодом.

4.8 Дизајн командног интерфејса

Командни интерфејс (енг. *Command Line interface*, CLI) је улазна тачку система. Дизајн примењује *Command Dispatcher* образац. Кориснички захтеви се делегирају сервисима за анализу или компилацију. Архитектура декомпонује систем на независне функционалне целине.

Систем подржава:

- интерактивни режим који обрађује појединачне захтеве и
- сервисни режим који омогућава комуникацију са развојним окружењима.

Архитектура садржи скуп подкоманди приказаних у табели 3.

Табела 3: Преглед доступних CLI команди

Команда	Опис	Параметри
<i>generate</i>	Генерација података	Улазни фајл, излаз, формат, семе
<i>compile</i>	Компилација модула	Улазни фајл, излаз
<i>check</i>	Валидација кода	Листа фајлова
<i>init</i>	Иницијализација пројекта	Директоријум, назив, примери
<i>info</i>	Метаподаци	Проширени приказ
<i>lsp</i>	Језички сервер	stdio, порт, лог фајл

Ово поглавље приказује имплементацију развијеног DSL-а и пратећег језичког сервера. Све компоненте система реализоване су у програмском језику *Rust*. Реализација прати фазе дефинисане у поглављу о дизајну. Лексички анализатор преводи изворни текст у ток токена (поглавље 5.1). Синтаксни анализатор трансформише ток токена у AST (поглавља 5.2 и 5.3). Компонента за семантичку анализу врши валидацију над генерисаним стаблом (поглавље 5.4). Систем преводи AST у HIR (поглавље 5.5).

Сваки модул развијен је као независна целина. Модули користе централизоване механизме за управљање грешкама и дијагностиком. Архитектура подржава размену података са језичким сервером. Исправност сваке компоненте потврђена је кроз јединичне и интеграционе тестове.

5.1 Лексичка анализа

Лексичка анализа представља прву фазу превођења. Циљ је трансформисати изворни текст у низ токена. Имплементација користи итератор *Peekable<CharIndices>*. Итератор омогућава обраду текста уз могућност прегледа наредних карактера без померања позиције показивача [31]. Увид у наредне карактере омогућава препознавање вишекарактерних оператора попут `&&`, `||`, `..` и `..=`. Препознавање се извршава без алокације додатног стања у меморији.

За препознавање кључних речи користи се регистар *KEYWORD_REGISTRY*. Регистар је имплементиран помоћу статичке мапе *once_cell::sync::Lazy* [31]. Мапа обезбеђује константно време претраге приликом идентификације резервисаних речи. Сваки токен чува позицију из изворног кода унутар структуре *Span*. Структура *Span* садржи почетну и крајњу линију и колону. Подаци о позицији омогућавају језичком серверу мапирање дијагностичких порука на оригинални текст.

Структура *LexerError* служи за мапирање лексичких неправилности у дијагностичке поруке. Лексички анализатор не прекида рад при наиласку на невалидан карактер или нетерминирани литерал. Компонента генерише грешку и наставља обраду преосталог дела датотеке. Јединични тестови у датотеци *tests.rs* потврђују исправност рада лексичког анализатора. Тестови покривају примитивне токене, сложене операторе и граничне случајеве попут недовршених стрингова. Лексички анализатор обезбеђује структуриране податке за потребе синтаксног анализатора.

5.2 Синтаксна анализа

Синтаксни анализатор трансформише низ токена у AST. Имплементација је подељена на засебне датотеке попут *expression.rs*, *statement.rs* и *token_stream.rs*. Централна структура *Parser* управља целокупним процесом синтаксне анализе.

5.2.1 Управљање током токена

Компонента *TokenStream* омогућава предиктивну анализу и вирење унапред (енг. *lookahead*) (листинг 12). Синтаксни анализатор прегледа наредне токене без њиховог уклањања из примарног тока. Преглед токена унапред неопходан је за разликовање литерала и типова података.

```
pub struct TokenStream<I>
where
    I: Iterator<Item = Result<Token, LexerError>>,
{
    lexer: Peekable<I>,
    buffer: VecDeque<Token>,
    last_span: Span,
}
```

Листинг 12: Структура *TokenStream* компоненте

TokenStream користи итератор *Peekable* и интерни бафер *VecDeque* за привремено складиштење токена. Архитектура омогућава управљање стањем тока и прескакање токена током синхронизације. Поље *last_span* служи за праћење последње познате позиције у изворном коду.

5.2.2 Синтаксна анализа израза

Обрада израза ослања се на Пратов (енг. *Pratt*) синтаксни анализатор. Функција *parse_expression* прихвата аргумент *precedence* за одређивање приоритета оператора. Логика парсирања подељена је на две фазе. Прва фаза користи позив функције *parse_primary_expression* за обраду атомских вредности попут литерала, идентификатора и заграда. Друга фаза обухвата итеративну проверу наредног токена и поређење са тренутним нивоом приоритета. Синтаксни анализатор позива функцију *parse_infix_expression* за операције над већ обрађеним изразима.

5.2.3 Синтаксна анализа исказа

Функција *parse_statement* усмерава ток извршавања на основу врсте токена. Функција позива метод *parse_directive* за кључне речи *output*, *seed* и *import*. Методи *parse_template* и *parse_struct* обрађују шаблоне и структуре. Методи *parse_enum* и *parse_type_declaration* парсирају енумерације и декларације типова. Искази представљају заокружене семантичке целине попут дефиниција структура или шаблона. Изрази се парсирају унутар блокова *ExpressionStatement* са обавезном тачком-зарезом на крају.

5.2.4 Синтаксна анализа типова података и ограничења

Синтаксни анализатор обрађује типове података кроз систем наменских функција. Функција *parse_type* препознаје примитивне типове, листе и кориснички дефинисане типове. Систем подржава додавање семантичких ограничења на типове података. Ограничења се синтаксички дефинишу унутар угластих заграда. Регистар *CONSTRAINTS* садржи листу свих дозвољених врста ограничења. Подржана ограничења укључују опсег, минималну вредност, максималну вредност и дужину. Регистар користи статичку мапу за валидацију идентификатора ограничења током парсирања. Непозната ограничења изазивају прекид парсирања тренутног израза и генерисање одговарајуће грешке.

5.2.5 Синтаксна анализа текстуалних шаблона

Језик подржава валидацију стрингова помоћу специјализованих текстуалних шаблона. Функција *parse_string_pattern* анализира садржај текстуалног литерала карактер по карактер. Синтаксни анализатор користи итератор *Peekable<CharIndices>* за навигацију кроз садржај шаблона. Механизам из литерала издваја константне делове и специјалне услове. Услови дефинишу правила попут понављања малих слова, великих слова или бројева. Број понављања карактера може бити дефинисан динамички. Динамички број понављања захтева инстанцирање секундарног лексичког и синтаксног анализатора. Секундарни синтаксни анализатор обрађује угњеждане изразе унутар угластих заграда шаблона.

5.2.6 Руковање грешкама и синхронизација

Опоравак од грешака реализован је кроз метод *synchronize*. Синтаксни анализатор наставља рад након појаве грешке *ParserError*. Компонента прескаче токене унутар *TokenStream*-а до следећег поузданог граничника. Као граничници служе тачка-зарез или почетак новог исказа. Синхронизација омогућава пријаву више независних синтаксних грешака током једног покретања.

5.2.7 Тестирање синтаксне анализе

Исправност синтаксног анализатора осигурана је кроз скуп аутоматизованих тестова. Јединични тестови проверавају рад синтаксног анализатора у изолованом окружењу. Изоловано тестирање потврђује правилну конструкцију AST-а за сваки појединачни исказ. Интеграциони тестови верификују правилну комуникацију између лексичког и синтаксног анализатора. Аутоматизовани тестови спречавају регресије током даљег развоја синтаксних правила.

5.3 Апстрактно синтаксно стабло

AST је имплементирано као модуларна архитектура унутар датотеке *mod.rs*. Систем је подељен на логичке целине за изразе, исказе, варијанте, поља и ограничења. Централни чвор за репрезентацију изрази у коду је структура *Expression*. Структура *Expression* и припадајућа енумерација *ExpressionKind* приказане су на листингу 13.

```
pub struct Expression {
    pub kind: ExpressionKind,
    pub span: Span,
}

pub enum ExpressionKind {
    IntLiteral(usize),
    StringLiteral(String),
    BooleanLiteral(bool),
    Identifier(String),
    List(Vec<Element>),
    Type(DataType),
    Reference {
        template: String,
        field: String,
    },
    // ... остале варијанте су изостављене ради прегледности
}
```

Листинг 13: Структура и енумерација за репрезентацију језичких изрази

Рекурзивни чворови за префиксне и инфиксне операторе омогућени су употребом паметних показивача (енг. *smart pointer*) *Box* [31]. Рад са листама имплементиран је преко структуре *Element*. Структура *Element* дозвољава придруживање тежинских изрази члановима листа. Структура програма и синтаксне наредбе представљене су кроз енумерацију *Statement*. Свака варијанта унутар енумерације *Statement* чува метаподатке о позицији и структури изворног кода. Искази који садрже самосталан израз издвајају се унутар наменске структуре *ExpressionStatement*. За потребе претраге, валидације и трансформације генерисаног стабла развијен је интерфејс *Visitor*. Интерфејс дефинише методе за посету сваког појединачног типа чвора у AST-у. Функције *walk_program*, *walk_statement* и *walk_expression* омогућавају рекурзивно кретање кроз дубину стабла. Помоћне функције усмеравају посетитеља на угњеждене подчворове изрази, варијанти и поља.

5.4 Семантичка анализа

Семантичка анализа врши контекстуалну валидацију AST-а након синтаксне анализе. Процес осигурава испуњеност правила која превазилазе могућности контекстно-слободних граматика. Архитектура система заснива се на вишефазном модуларном приступу. Имплементација користи дизајнерски шаблон посетилац за обилазак језичких конструкција. Структура *SemanticAnalyser*<'a> оркестрира процес анализе кроз методу *analyse_with_imports*. Систем као резултат генерише структуру *AnalysisResult*. Излазна структура садржи табелу симбола, мапу референци, мапу типова и колекцију дијагностичких порука.

5.4.1 Фазе семантичке анализе

Процес семантичке анализе обухвата пет секвенцијалних фаза.

1. Прва фаза подразумева изградњу табеле симбола кроз структуру *SymbolTableBuilder*. Компонента региструје глобалне и локалне симболе уз успостављање хијерархије опсега важења. Детекција дуплираних декларација прекида процес анализе пре покретања наредних фаза.
2. Друга фаза извршава детекцију циклуса у наслеђивању шаблона. Систем проверава постојање кружних зависности пре почетка валидације поља и референци.
3. Трећа фаза обухвата проверу референци помоћу компоненте *ReferenceChecker*. Анализатор потврђује постојање сваког идентификатора у тренутном контексту или унутар увезених модула.
4. Четврта фаза мапира међусобне везе објеката кроз структуру *ReferenceTracker*. Прикупљени метаподаци служе за очување референцијалног интегритета током генерисања података.
5. Пета фаза извршава проверу типова и ограничења помоћу компоненте *TypeChecker*. Систем закључује типове израза, детектује некомпатибилност оператора и валидира семантичку исправност ограничења.

5.4.2 Табела симбола и управљање опсезима важења

Структура *SymbolTable* управља симболима кроз организацију стабла повезаних опсега. Сваки опсег садржи идентификатор *ScopeId*, мапу локалних симбола и показивач ка родитељском опсегу. Лексичко претраживање симбола од локалног ка глобалном нивоу извршава се позивом методе *lookup*.

Систем дефинише врсте опсега кроз енумерацију *ScopeKind*. *Global* је врховни опсег за декларације шаблона, структура, енумерација и типова. *Template* дефинише опсег унутар једног шаблона. *Enum* ограничава контекст унутар дефиниције енумерације. *Generate* означава опсег унутар блока за генерацију података. *Directive* и *Block* служе као помоћни локални опсези.

Симболи су представљени структуром *Symbol*. Полиморфна структура за чување метаподатака о језичким конструктима приказана је на листингу 14.

```
pub enum SymbolKind {
    Template {
        fields: Vec<String>,
        parent: Option<String>,
    },
    Struct {
        name: String,
        fields: Vec<Field>,
    },
    Enum {
        variants: Vec<VariantInfo>,
    },
    TypeAlias {
        name: String,
        data_type: Expression,
    },
    Field {
        template_name: String,
        is_override: bool,
        expression: Expression,
    },
}
```

Листинг 14: Енумерација *SymbolKind* за репрезентацију семантичких метаподатака

5.4.3 Детекција циклуса у наслеђивању

Структура *SymbolTable* имплементира алгоритам за детекцију циклуса унутар методе *check_inheritance_cycle*. Алгоритам спречава бесконачну рекурзију приликом кружног наслеђивања ша-

блона. Претрага пролази кроз ланац наслеђивања праћењем референци ка родитељима. Алгоритам користи хеш-скуп за праћење посећених чворова. Поновно појављивање истог шаблона унутар тренутне путање прекида извршавање претраге. Метод враћа листу стрингова са тачним редоследом цикличне зависности. Аналитатор трансформише резултат у дијагностичку грешку *SemanticError::InheritanceCycle*.

5.4.4 Провера типова и семантичких ограничења

Компонента *TypeChecker* извршава валидацију типова израза унутар AST-а. Модул имплементира интерфејс посетитеља за аутоматско пресретање и анализу чворова стабла.

Компонента извршава валидацију тежинских фактора унутар дефиниција енумерација. Метод *visit_variant* захтева целобројни тип израза за дефинисање тежине. Одступање од целобројног типа резултује грешком о неслагању типова. Аналитатор контролише команде за генерацију података кроз метод *visit_generate*. Израз за дефинисање броја генерисаних инстанци мора припадати целобројном типу. Систем анализира изразе ограничења попут *Range*, *Min*, *Max*, *Length* и *MultipleOf*. Семантички аналитатор проверава компатибилност ограничења са тренутним типом података.

5.4.5 Дијагностика и руковање грешкама

Семантички аналитатор спроводи стратегију акумулације грешака уместо прекидања извршавања при првој неправилности. Систем чува све пронађене нерегуларности унутар колекције грешака структуре *SemanticAnalyser*.

Семантичке грешке су типизирани кроз енумерацију *SemanticError*. Дефиниција типизираних семантичких грешака и метод за њихову трансформацију у дијагностички формат приказани су на листингу 15.

```
pub enum SemanticError {
    UnknownIdentifier {
        name: String,
        span: Span,
    },
    TypeMismatch {
        expected: String,
        found: String,
        span: Span,
    },
    // ... остале варијанте грешака
}

impl SemanticError {
    pub fn to_diagnostic(&self) -> Diagnostic {
        match self {
            SemanticError::TypeMismatch { expected, found, span } => {
                Diagnostic::error(
                    *span,
                    format!("Type mismatch: expected '{}', found '{}'", expected, found),
                )
                .with_code(DiagnosticCode::TypeMismatch)
                .with_hint("Ensure the expression matches the expected type")
            }
            // ... мапирање осталих грешака
        }
    }
}
```

Листинг 15: Дефиниција семантичких грешака и мапирање у објекте дијагностике

Свака варијанта садржи структуру *Span* за мапирање координата у изворном коду. Метода *to_diagnostic* трансформише објекат семантичке грешке у LSP дијагностички формат. Систем генерише контекстуалне савете за исправку на основу специфичног типа грешке.

5.5 Високонивојска међурепрезентација

HIR је упрошћена форму програма након завршене семантичке анализе. Трансформација из AST-а у HIR уклања синтаксне конструкције и нормализује структуру података. HIR служи као основа за даљу оптимизацију, проверу компатибилности и серијализацију модула. Инфраструктура за управљање меморијом и дијагностиком приказана је на листингу 16.

```
pub struct StringPool {
    strings: Vec<String>,
    map: HashMap<String, StringId>,
}

pub struct SpanMap<K, V> {
    entries: Vec<(K, V)>,
    cache: OnceLock<HashMap<K, V>>,
}
```

Листинг 16: Инфраструктура за меморијску оптимизацију и мапирање изворног кода

Интернирање стрингова преко структуре *StringPool* обезбеђује репрезентацију идентификатора. *SpanMap* омогућава повезивање метаподатака са позицијама у изворном коду. Структура користи *OnceLock* за лењо кеширање претрага ради убрзања приступа координатама. Поступак трансформације изводи компонента *AstLowering*. Имплементација структуре *AstLowering* приказана је на листингу 17.

```
pub struct AstLowering {
    context: LoweringContext,
    string_pool: StringPool,
    source_map_builder: SourceMapBuilder,
    directives: Vec<Directive>,
    source_file: PathBuf,
    items: Vec<Item>,
}
```

Листинг 17: Структура *AstLowering* задужена за трансформацију AST-а у HIR

Компонента *AstLowering* користи *string_pool* за дедупликацију текста и *source_map_builder* за праћење порекла чворова. Метода *lower_expr* рекурзивно обилазе чворове AST-а. Генерисани модел омогућава бинарну серијализацију модула.

5.6 Систем модула

Систем модула омогућава декомпозицију програма у независно преводиве целине. Сваки модул представља јединицу компилације која се након анализе серијализује у бинарни формат *.tmod*. Архитектура подржава увоз симбола кроз систем референци и мапирање идентификатора.

5.6.1 Глобално адресирање и серијализација

Физичка локација ставки апстрахована је коришћењем униформног глобалног адресирања. Идентификација симбола у меморији ослања се на композитне нумеричке кључеве (листинг 18).

```
#[derive(Copy, Clone, Hash, Eq, PartialEq)]
pub struct LocalItemId(pub u32);

#[derive(Copy, Clone, Hash, Eq, PartialEq)]
pub struct ModuleId(pub u32);

#[derive(Copy, Clone, Hash, Eq, PartialEq)]
pub struct GlobalItemId {
    pub module: ModuleId,
    pub item: LocalItemId,
}
```

Листинг 18: Структуре података за глобално адресирање ставки

Структура *LocalItemId* одређује ставку унутар граница једне датотеке. Структура *GlobalItemId* спаја локални идентификатор са идентификатором модула (*ModuleId*). Глобално адресирање омогућава униформан рад са локалним и спољашњим симболима.

Бинарни формат модула остварен је коришћењем библиотеке *rpm_serde* [32]. *Rpm_serde* омогућава компактну серијализацију *Rust* структура у *MessagePack* формат. Нумерички кључеви се пресликавају у бинарне репрезентације, што смањује заузеће простора на диску.

5.6.2 Резолуција, учитавање и релокација

Компонента *ModuleResolver* управља процесом проналажења и учитавања зависности у меморију. Претрага обухвата директоријуме стандардне библиотеке и локалне путање пројекта. Имплементација методе за разрешавање зависности приказана је на листингу 19.

```
pub struct ModuleResolver {
    search_paths: Vec<PathBuf>,
    cache: HashMap<String, Module>,
    next_module_id: u32,
}

impl ModuleResolver {
    fn resolve_one(
        &mut self,
        name: &str,
        source_path: &Path
    ) -> Result<(), ResolveError> {
        if self.cache.contains_key(name) { return Ok(()); }

        let path = self.find_tmod(name, source_path)?;
        let mut module = Module::load(&path)?;
        let allocated_id = ModuleId(self.next_module_id);
        self.next_module_id += 1;
        self.cache.insert(name.to_string(), module);

        Ok(())
    }
}
```

Листинг 19: Логика разрешавања, валидације кеша и релокације модула

ModuleResolver користи хеш-мапу за мемоизацију. Мемоизација спречава поновно учитавање истог модула. У случају неуспешне резолуције или грешке при читању, систем генерише *ResolveError* са подацима о узроку.

5.6.3 Серијализација и интегритет података

Бинарни формат модула остварен је коришћењем библиотеке *rpm_serde* [32]. *Rpm_serde* омогућава брзу и компактну серијализацију у *MessagePack* формат. Модули садрже метаподатке о увозима и дефинисаним ставкама. Апстракција *ItemRef* обезбеђује интегритет референци између модула.

Структуре *ItemId* и *StringId* дефинишу идентификаторе. Централизован *StringPool* управља *StringId* вредностима. *StringPool* елиминише дуплирање стрингова и убрзава њихово поређење. *ItemRef* повезује контекст модула преко *StringId* са *ItemId* ставке. Статичко разрешавање зависности претходи генерисању кода.

5.7 Генерисање података

Генерисање података ослања се на концепт лење евалуације (енг. *lazy evaluation*). Структура *RecordGenerator* имплементира стандардни интерфејс итератора [31]. Итератор генерише записе секвенцијално на експлицитан захтев. Систем обрађује листу директива и конструише појединачне записе преко функције *generate_record*. Алгоритам користи интерни евалуатор за динамичко израчунавање вредности поља током итерације. Имплементација елиминише потребу за учитавањем комплетног скупа података у радну меморију.

Особина (енг. trait) *FileGenerator* (листинг 20) дефинише апстракцију за серијализацију генерисаних структура. Интерфејс захтева дефинисање екстензије датотеке и имплементацију методе за генерисање појединачног записа. Опционе методе омогућавају додавање заглавља, подножја и сепаратора између записа. Систем изворно подржава упис у CSV, JSON, XML и SQL формате кроз засебне имплементације дефинисане особине. Сваки формат подржава динамичку инстанцијацију путем конфигурационе мапе објеката. Конфигурација омогућава прилагођавање атрибута попут сепаратора, увлачења текста или имена SQL табеле. Функција *write_records* преузима итератор и уписује форматиране линије у излазну датотеку на диску.

```
pub type Record = IndexMap<String, Object>;
pub trait FileGenerator: fmt::Debug {
    fn generate(&self, record: &Record) -> Result<String, GeneratorError>;
    fn extension(&self) -> &str;
    fn generate_header(&self, fields: &[String]) -> Option<String> { None }
    fn generate_footer(&self) -> Option<String> { None }
    fn needs_separator(&self) -> bool { false }
    fn separator(&self) -> &str { "" }
}
```

Листинг 20: Дефиниција особине *FileGenerator* за генерисање излазних формата.

5.8 Језички сервер

Комуникација са развојним окружењима одвија се кроз LSP. Сервер омогућава анализу изворног кода, навигацију кроз дефиниције и преглед упозорења у реалном времену. Имплементација користи библиотеку *tower-lsp*.

Структура *Workspace* управља документима. Приступ садржају датотека обезбеђује механизам за синхронизацију *RwLock*. Документ обједињује текст, верзију и структуру *Analysis*. Анализа садржи AST, табелу симбола, референце и листу дијагностичких порука.

Компонента *AnalysisEngine* координише превођење:

- лексичку и синтаксну анализу,
- резолуцију увоза модула кроз *ModuleResolver* и
- семантичку анализу путем *SemanticAnalyser-a*.

Метода *update* (листинг 21) повезује фазе компилације са LSP догађајима. Анализа се покреће при свакој измени текста. Резултати се мапирају у структуру документа, док се дијагностичке поруке преводе у LSP

формат. Раздвајање логике анализе од приказивања омогућава приказ грешака у едитору без прекида рада сервера.

```
pub async fn update(
    &self,
    uri: Url,
    text: String,
    version: i32
) -> Vec<lsp_types::Diagnostic> {
    let file_path = uri.to_file_path().ok();
    let result = AnalysisEngine::analyse(&text, file_path.as_deref());

    let document = match (result.ast, result.symbol_table) {
        (Some(ast), Some(symbols)) => {
            Document::new(uri.clone(), text, version).with_analysis(Analysis {
                // ... креирање анализе
            })
        }
        _ => Document::new(uri.clone(), text, version),
    };

    self.insert(uri.clone(), document).await;

    result.diagnostics.into_iter()
        .map(Diagnostic::to_lsp_diagnostics)
        .collect()
}
```

Листинг 21: Ажурирање документа и синхронизација анализе

5.9 Командни интерфејс

Реализација CLI-а користи библиотеку *clap* [33]. Библиотека омогућава декларативно дефинисање аргумената кроз *Rust* структуре и енумерације. Мапирање корисничког улаза у типске вредности елиминира потребу за ручним парсирањем стрингова. Систем детектује неисправне уносе током фазе превођења кода, што повећава поузданост програма.

Архитектура почива на енумерацији *Command*. Енумерација дефинише све доступне операције система. Свака варијанта поседује структуру аргумената. Дизајн омогућава јасну декомпозицију логике (листинг 22).

```
#[derive(Parser)]
#[command(name = "testa")]
pub struct Cli {
    #[command(subcommand)]
    pub command: Command,
}

#[derive(Subcommand)]
pub enum Command {
    Generate { ... },
    Compile { ... },
    Init { ... },
    // ... остале CLI команде
}
```

Листинг 22: Структура CLI команди

Функција *run* координише извршавање. Систем акумулира *Diagnostic* поруке приликом појаве грешака. Поруке се форматирају за корисника. Програм враћа повратни код за интеграцију са системским алатима.

У овом поглављу представљени су резултати испитивања перформанси развијених програмских решења. Анализа обухвата:

- временску ефикасност фаза превођења (поглавље 6.2),
- утицај система модула на брзину превођења (поглавље 6.3),
- временску и меморијску ефикасност генерисања података (поглавље 6.4) и
- време одзива језичког сервера (поглавље 6.5).

6.1 Експериментално окружење

Мерења су спроведена у контролисаном хардверском и софтверском окружењу ради добијања репродуктивних резултата. Сви тестови перформанси извршени су на рачунару конфигурације:

- Процесор: Intel Core i7-11800H радног такта 2,30 GHz
- Радна меморија: 16 GB DDR4
- Оперативни систем: Windows 10 Professional

Тестирање временске ефикасности процеса превођења и језичког сервера реализовано је применом библиотеке *Criterion* унутар програмског језика *Rust*. Мерења извршена су над програмом компајлираним у *release* режиму. *Criterion* користи методу статистичког узорковања са претходним фазама загревања како би се ублажио утицај позадинских процеса оперативног система и стања кеш меморије. Свако појединачно мерење извршено је кроз стотину узастопних понављања.

6.2 Евалуација процеса превођења

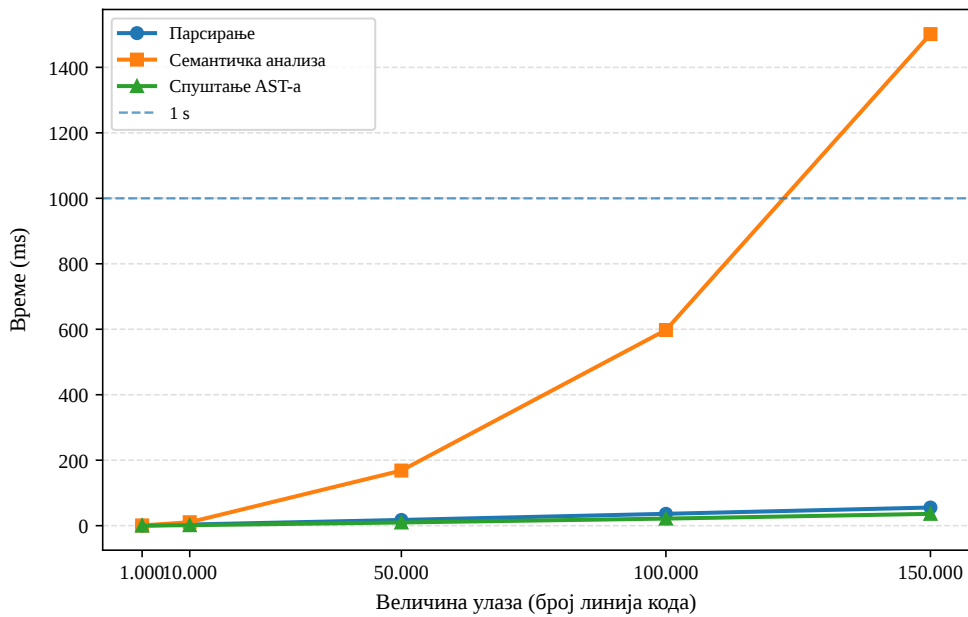
Евалуација процеса превођења спроведена је ради утврђивања времена потребног за извршавање појединачних фаза компилације при раду са улазним датотекама различитих величина. Тестирање је обухватило синтаксно исправне датотеке величине од 1.000 до 150.000 линија кода. Испитиване су:

- лексичка и синтаксна анализа (формирање стабла апстрактне синтаксе),
- семантичка анализа,
- процес спуштања (превођење у HIR).

Критеријум прихватљивости захтева укупно време превођења испод 1.000 ms за датотеке величине до 100.000 линија кода. Испуњење критеријума омогућава несметан интерактивни рад унутар развојних окружења, где језички сервер пружа брзу повратну информацију без приметног кашњења [6]. Непожељан исход представља прекорачење границе од 1.000 ms за средње величине датотека. Прекорачењем границе се нарушава континуитет рада корисника услед кашњења при анализи кода. Средње време извршавања фаза компилације садржи табела 4. График на слици 12 приказује зависност времена од величине датотеке.

Табела 4: Средње време извршавања фаза процеса превођења

Број линија кода	Формирање AST (ms)	Семантика (ms)	Спуштање (ms)	Укупно (ms)
1.000	0,40	0,69	0,18	1,27
10.000	3,50	10,58	1,50	15,58
50.000	17,51	168,35	9,75	195,61
100.000	36,23	597,76	21,59	655,58
150.000	55,46	1501,70	36,14	1593,30



Слика 12: Зависност времена извршавања фаза превођења од броја линија кода

Мерења указују на испуњење дефинисаног критеријума успеха за датотеке обима до 100.000 линија кода. Укупно време превођења датотеке од 50.000 линија износи 195,61 ms. Време превођења датотеке од 100.000 линија износи 655,58 ms. Измерена времена налазе се испод дефинисане границе од 1.000 ms. Лексичка и синтаксна анализа показују линеарни раст времена извршавања у односу на величину улаза. Синтаксна анализа датотеке обима 150.000 линија завршава се за 55,46 ms. Процес спуштања за датотеку обима 150.000 линија захтева 36,14 ms.

Укупно време превођења за датотеку од 150.000 линија износи 1593,30 ms. Измерена вредност прелази границу од 1.000 ms дефинисану за интерактивни рад. Прекорачење времена извршавања последица је нелинеарне сложености унутар фазе семантичке анализе. Време семантичке провере расте са 597,76 ms за 100.000 линија на 1501,70 ms за 150.000 линија. Добијени резултат дефинише границу употребљивости тренутне имплементације. Идентификовано ограничење указује на потребу за развојем алгорита за инкременталну семантичку анализу у оквиру будућег рада.

6.3 Евалуација система модула

Евалуација система модула квантификује трошак механизма за разрешавање зависности. Трошак обухвата читавање датотека и динамичку релокацију симбола. Повећани број улазно-излазних операција генерише оптерећење система. Критеријум успешности је линеарна сложеност $\mathcal{O}(N)$ при читавању N јединица компилације. Непожељан исход је експоненцијални раст времена превођења.

Експеримент је анализирао три топологије стабла зависности у величинама од 10, 50 и 100 јединица:

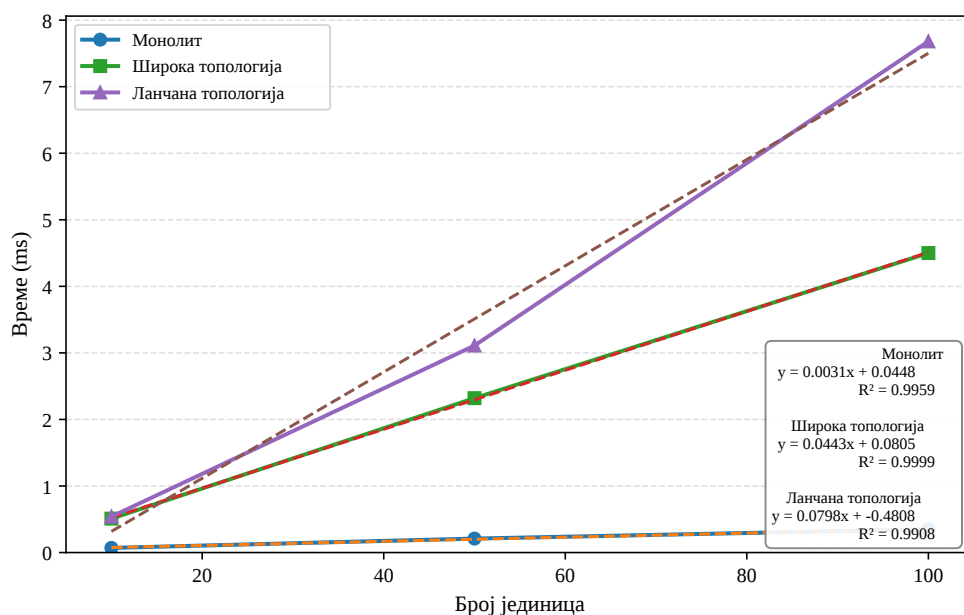
- монолитна база, једна датотека садржи све дефиниције. Мерење изолује брзину десеријализације без спољних датотека;
- широка топологија, главни модул увози N независних модула. Дубина стабла износи један. Мерење изолује трошак отварања датотека; и
- ланчана топологија, сваки модул увози тачно једну наредну јединицу. Дубина стабла износи N . Топологија оптерећује рекурзивни алгоритам за релокацију.

Резултати евалуације су приказани у табели 5.

Табела 5: Време учитавања и разрешавања зависности у односу на топологију модула

Број јединица	Монолит	Широка топологија	Ланчана топологија
10	0,07 ms	0,51 ms	0,54 ms
50	0,21 ms	2,32 ms	3,11 ms
100	0,35 ms	4,50 ms	7,68 ms

Подаци потврђују линеарно скалирање алгоритма. Монолитна база учитава 100 ставки за 0,35 ms. Резултат рефлектује извршавање једног системског позива. Широка топологија учитава 100 јединица за 4,50 ms. Разлика у времену представља трошак отварања појединачних датотека. Ланчана топологија захтева 7,68 ms за 100 јединица. График на слици 13 приказује линеарни тренд времена превођења у функцији броја модула. Линије регресије за сваку топологију потврђују да систем одржава константан однос раста у односу на број улазних јединица. Одступања података од линије регресије су занемарљива, што потврђује претпостављену линеарну сложеност $\mathcal{O}(N)$.



Слика 13: Линеарна зависност времена превођења од броја модула за различите топологије.

6.4 Евалуација генерисања података

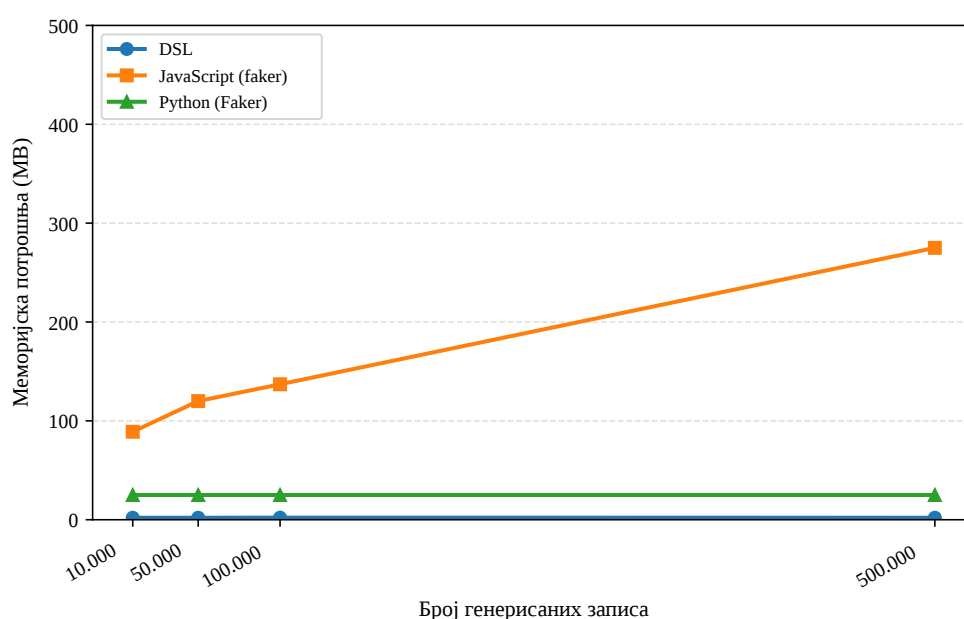
Евалуација генерисања података квантификује пропусну моћ и меморијску ефикасност система. Критеријум успешности имплементације захтева задржавање просторне сложености на нивоу $\mathcal{O}(1)$. Временска сложеност процеса генерисања мора имати линеарни раст $\mathcal{O}(N)$.

Експеримент пореди развијени систем са референтним библиотекама у језицима *Python (Faker)* (верзија 3.12.3) и *JavaScript (@faker-js/faker)* (NodeJS верзија 18.19.1). Тест модел представља равну структуру од десет поља различитих типова података. Одсуство релационих зависности између записа омогућава изолацију перформанси лење евалуације. Мерење обухвата генерисање скупа од 10.000, 50.000, 100.000 и 500.000 CSV записа. Меморијска потрошња се бележи на нивоу оперативног система праћењем максималне величине резидентног скупа (енг. *Peak RSS*). Резултати тестирања приказани су у табели 6.

Табела 6: Поређење времена генерисања и меморијске потрошње различитих имплементација

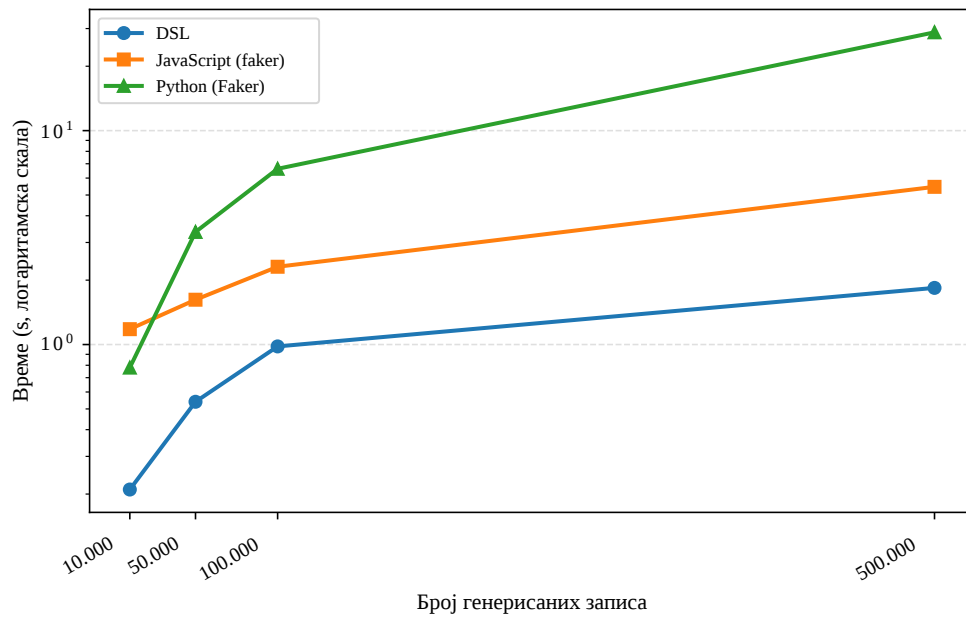
Број записа	Време (DSL)	Мем. (DSL)	Време (JS)	Мем. (JS)	Време (Python)	Мем. (Python)
10.000	0,21 s	2,0 MB	1,18 s	89 MB	0,78 s	25 MB
50.000	0,54 s	2,0 MB	1,62 s	120 MB	3,36 s	25 MB
100.000	0,98 s	2,1 MB	2,31 s	137 MB	6,63 s	25 MB
500.000	1,84 s	2,0 MB	5,46 s	275 MB	28,81 s	25 MB

Експериментални резултати потврђују боље перформансе развијеног система у поређењу са референтним скриптним језицима. Развијени систем задржава потрошњу меморије од 2 MB при генерисању 500.000 записа. Статична потрошња доказује просторну сложеност $\mathcal{O}(1)$ [30]. Имплементација у *Node.js* окружењу бележи линеаран раст меморије. Системска потрошња достиже 275 MB при генерисању 500.000 записа. График на слици 14 приказује меморијску зависност од броја записа.



Слика 14: Зависност меморијске потрошње од броја генерисаних записа.

Развијени систем остварује најмање време извршавања у свим сценаријима. Генерисање 500.000 записа траје 1,84 s. Резултат потврђује ефикасност директног уписа на диск без посредничких структура података. Време извршавања *Node.js* скрипте при генерисању 10.000 записа износи 1,18 s. *Python* имплементација задржава стабилну меморијску потрошњу од 25 MB. Перформансе *Python* скрипте линеарно опадају при повећању обима података. Време извршавања скупа података од 500.000 записа у *Python* окружењу износи 28,81 s. График на слици 15 приказује зависност времена од броја записа.



Слика 15: Зависност времена генерисања од броја записа.

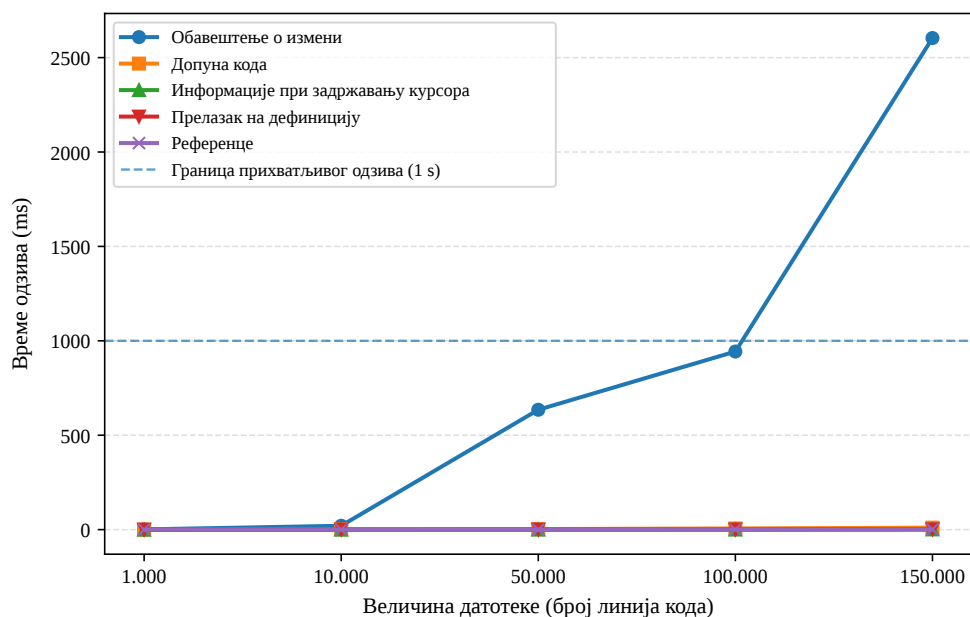
6.5 Евалуација језичког сервера

Циљ евалуације језичког сервера био је испитивање утицаја величине датотеке на време одзива језичког сервера. Пожељан исход је време одзива мање од 100 ms. Временски оквир мањи од 100 ms омогућава кориснику осећај тренутне реакције система [34]. Граница прихватљивости износи 1 s, након чега долази до прекида тока мисли корисника [34]. Непожељан исход је време одзива дуже од 1 s.

Експеримент је спроведен мерењем времена одзива LSP захтева унутар контролисаног тестног окружења. Тестирање је извршено над пет синтаксно исправних DSL датотека различитих величина. Величине датотека износиле су 1.000, 10.000, 50.000, 100.000 и 150.000 линија кода. За сваку датотеку је измерено време одзива пет различитих операција језичког сервера. Анализиране операције обухватале су:

- измену документа,
- приказ информација,
- прелазак на дефиницију,
- претрагу референци и
- аутоматско допуњавање кода.

Резултати мерења приказани су на слици 16. Прикупљени подаци указали су на постојање две категорије перформанси. Категорије су дефинисане на основу архитектонске реализације операција. Прву категорију су чиниле функционалности засноване на претрази табеле симбола. Операције су задржале константно време одзива од приближно 1 ms на целом опсегу тестирања. Друга категорија је обухватала операцију измене документа која захтева поновну обраду текста. Поновна обрада текста је показала линеаран раст времена одзива са повећањем величине датотеке. При раду са датотеком од 1.000 линија кода време одзива је износило 1,29 ms. На датотеци од 150.000 линија кода време одзива достигло је 2.603,50 ms.



Слика 16: Зависност времена одзива LSP операција од величине датотеке.

Синтеза спроведене евалуације потврдила је зависност укупних перформанси од примењеног начина синтаксне анализе. Операције над табелом симбола задржале су високу ефикасност кроз цео експеримент. Време одзива на датотеци од 150.000 линија кода премашило је прихватљив праг одзива од 1 s. Синтаксна анализа је узрок пада скалабилности архитектуре. Имплементација алгоритма за инкременталну синтаксну анализу је наредни корак у развоју алата.

6.6 Дискусија резултата

Резултати евалуације потврђују ефикасност развијеног система. Архитектура испуњава критеријуме прихватљивости за датотеке до 100.000 линија кода. Развијено решење превазилази перформансе референтних алата при генерисању података.

Директан упис на диск одржава константну потрошњу меморије. Систем заузима 2 MB простора независно од обима излазних података. Добијена вредност потврђује просторну сложеност $\mathcal{O}(1)$. Механизам модула линеарно скалира. Операције језичког сервера над табелом симбола остварују време одзива од 1 ms. Измерене вредности омогућавају несметан рад корисника у развојном окружењу.

Тестирање открива ограничења архитектуре. Обрада датотека већих од 100.000 линија деградира перформансе. Време извршавања семантичке провере расте нелинеарно. Операција измене документа захтева синтаксну анализу текста при сваком уносу карактера. Одзив језичког сервера прелази границу од 1 s за масивне датотеке.

Уочена ограничења дефинишу правац даљег развоја система. Оптимизација перформанси захтева инкрементални модел обраде AST-а. Инкрементална анализа представља надоградњу за рад са великим монолитним датотекама. Систем успешно интегрише концепте компилаторске теорије и генератора података.

У оквиру рада дизајниран је и имплементиран екстерни DSL за генерисање тест података и пратећи језички сервер. Анализа постојећих решења показала је да алати за синтезу података захтевају писање императивног кода или ослањање на затворене графичке платформе које ограничавају локално извршавање и верзионисање кода. Решење превазилази ограничења увођењем декларативне текстуалне синтаксе.

Технички доприноси рада обухватају:

- развој преводиоца у програмском језику *Rust* са лексичком, синтаксном и семантичком анализом;
- успостављање модуларне архитектуре засноване на HIR и бинарној серијализацији ради убрзања компајлације;
- имплементацију стохастичког генератора података који подржава статистичке расподеле и очување референцијалног интегритета путем тополошког сортирања;
- развој језичког сервера за интеграцију са развојним окружењима путем LSP-а.

Дизајн доказује могућност креирања специјализованог алата. Алат пружа функционалности статичке анализе у реалном времену и задржавање перформансе приликом локалног извршавања.

Архитектура система поставља основу за проширења. Модуларни дизајн предњег и задњег дела преводиоца омогућава идентификацију праваца за развој.

8.1 Унапређење система компилације

Тренутна имплементација захтева ручно покретање преводаца за сваки модул. Будућа верзија система треба да уведе механизам за инкременталну компилацију који би аутоматизовао процес. Систем би пратио измене у изворним *.testa* датотекама помоћу пратилаца датотека (енг. *file watchers*) и поредио временске ознаке (енг. *timestamps*) изворног кода са постојећим бинарним модулима.

Приликом детекције измене, преводац би ажурирао само оне модуле који су измењени. Интеграција детекције измене уклонила би потребу за мануелном интервенцијом и омогућила конзистентност између изворног и преведеног кода током развоја.

8.2 Средњенивојска међурепрезентација

Тренутна архитектура преводи процесни низ директно из AST структуре у HIR. Имплементација MIR-а омогућила би увођење додатних фаза оптимизације кода. MIR служи као канонички приказ програма где се сложени изрази декомпонују у низ једноставних, линеарних операција.

Увођење MIR-а омогућава имплементацију анализа попут анализе тока података (енг. *data-flow analysis*) или оптимизације „мртвог кода“ (енг. *dead code elimination*). Прелазак на двостепену IR (HIR за семантичку анализу, MIR за оптимизацију) доприноси модуларности преводиоца. Процес генерисања излазног кода постаје независан од специфичности AST-а, што олакшава додавање нових бекенд циљева (нпр. инфраструктура за генерисање машинског кода (енг. *Low-Level Virtual Machine*) или генерација C кода).

Списак слика

Слика 1	Трансформација израза $5 + 2 * 3$ у AST на основу ВР-а оператора	4
Слика 2	Структура дизајнерског шаблона посетилац [12]	6
Слика 3	Поређење AST-а и HIR-а на примеру декларације шаблона.	7
Слика 4	Процес серијализације компајлираног модула и његовог увоза током компајлације.	8
Слика 5	Теоријски приказ униформне (лево) и категоријалне расподеле (десно).	10
Слика 6	Комуникација између <i>едитора</i> и језичког сервера током уређивања документа [3].	11
Слика 7	GUI платформе <i>Mockaroo</i> за конфигурацију генератора података	16
Слика 8	Дијаграм архитектуре система	17
Слика 9	Ланац обраде у компајлеру	18
Слика 10	Архитектура LSP система и ток обраде LSP захтева.	23
Слика 11	Секвенцијални дијаграм обраде догађаја при отварању датотеке	23
Слика 12	Зависност времена извршавања фаза превођења од броја линија кода	38
Слика 13	Линеарна зависност времена превођења од броја модула за различите топологије.	39
Слика 14	Зависност меморијске потрошње од броја генерисаних записа.	40
Слика 15	Зависност времена генерисања од броја записа.	41
Слика 16	Зависност времена одзива LSP операција од величине датотеке.	42

Списак листинга

Листинг 1	Дефиниција граматике у алату <i>textX</i>	13
Листинг 2	Пример <i>JSON Schema</i>	14
Листинг 3	Шема поруке унутар <i>proto</i> датотеке	15
Листинг 4	Имплементација псеудослучајног генератора у језику <i>Python</i>	15
Листинг 5	Пример глобалних директива и увоза модула	18
Листинг 6	Дефинисање скаларних типова са ограничењима	18
Листинг 7	Дефинисање колекцијских типова и вишедимензионалних структура	19
Листинг 8	Енумерација са факторима тежине расподеле	19
Листинг 9	Структурна композиција и механизам наслеђивања	19
Листинг 10	Референцирање вредности ради очувања интегритета	20
Листинг 11	Концептуални модел HIR модула	21
Листинг 12	Структура <i>TokenStream</i> компоненте	28
Листинг 13	Структура и енумерација за репрезентацију језичких израза	29
Листинг 14	Енумерација <i>SymbolKind</i> за репрезентацију семантичких метаподатака	30
Листинг 15	Дефиниција семантичких грешака и мапирање у објекте дијагностике	31
Листинг 16	Инфраструктура за меморијску оптимизацију и мапирање изворног кода	32
Листинг 17	Структура <i>AstLowering</i> задужена за трансформацију AST-a у HIR	32
Листинг 18	Структуре података за глобално адресирање ставки	33
Листинг 19	Логика разрешавања, валидације кеша и релокације модула	33
Листинг 20	Дефиниција особине <i>FileGenerator</i> за генерисање излазних формата.	34
Листинг 21	Ажурирање документа и синхронизација анализе	35
Листинг 22	Структура CLI команди	35

Списак табела

Табела 1 Пресликавање локалних идентификатора у глобални адресни простор током релокације.	8
Табела 2 Поређење предложеног решења са постојећим алатима	16
Табела 3 Преглед доступних CLI команди	25
Табела 4 Средње време извршавања фаза процеса превођења	37
Табела 5 Време учитавања и разрешавања зависности у односу на топологију модула	39
Табела 6 Поређење времена генерисања и меморијске потрошње различитих имплементација	40

Списак коришћених скраћеница

Скраћеница	Опис
API	Application Programming Interface (програмски интерфејс апликације)
AST	Abstract Syntax Tree (апстрактно синтаксно стабло)
BP	Binding Power (снага везивања)
CLI	Command Line Interface (интерфејс командне линије)
DSL	Domain Specific Language (језик специфичан за домен)
GPL	General Purpose Language (језик опште намене)
GUI	Graphical User Interface (графички кориснички интерфејс)
HIR	High-level Intermediate Representation (високонивојска међурепрезентација)
IR	Intermediate Representation (међурепрезентација)
JSON	JavaScript Object Notation (текстуални формат за размену података)
LIR	Low-level Intermediate Representation (нисконивојска међурепрезентација)
LSP	Language Server Protocol (протокол језичких сервера)
MIR	Medium-level Intermediate Representation (средњенивојска међурепрезентација)
PRNG	Pseudo-Random Number Generator (псеудослучајни генератор бројева)
RPC	Remote Procedure Call (позивање удаљених процедура)
XML	Extensible Markup Language (прошириви језик за означавање података)

Списак коришћених појмова

Појам	Објашњење
Двоструко упућивање	Механизам избора методе у извршном времену на основу типова два објекта: објекта који прима позив и објекта који се прослеђује као аргумент.
Засебна компајлација	Метод превођења програмског кода где се изворне датотеке компајлирају независно, чиме се убрзава развој јер нема потребе за поновним превођењем целог система.
Инкрементална компајлација	Техника поновног превођења само измењених делова кода ради скраћивања времена изградње система.
Интернирање стрингова	Техника оптимизације меморије где се јединствени текстуални низови складиште у базен и мењају нумеричким кључевима ради бржег поређења.
JSON-RPC	Протокол за позивање удаљених процедура који користи JSON формат за кодирање порука, коришћен у LSP комуникацији.
Међурепрезентација	Интерни модел или структура података коју преводилац користи за представљање изворног кода између фазе синтаксне анализе и генерисања кода.
Повезивање	Процес спајања независних модула у извршну целину.
Релокација	Корак током повезивања који ажурира нумеричке референце и меморијске адресе унутар уčitаног кода.
Самоописујући формат	Формат записа који не захтева унапред дефинисану шему, већ унутар записа чува метаподатке о типовима и структурама поља.
Серијализација	Процес превођења меморијских структура података у линеарни низ бајтова погодан за трајно чување на диску или пренос мрежом.
Спуштање	Процес сукцесивне трансформације међурепрезентације којим се сложени језички конструкти свode на једноставније, каноничке форме ближе извршном коду.
Табела симбола	Структура података коју преводилац користи за мапирање идентификатора са својствима, опсезима видљивости и локацијама.

Биографија

Лазар Нагулов је рођен 30. априла 2003. године у Сомбору. Основну школу „Доситеј Обрадовић” завршио је у родном граду. Након тога уписао је гимназију „Вељко Петровић” у Сомбору, смер за ученике са посебним способностима за рачунарство и информатику, коју је завршио 2022. године. Исте године уписао је Факултет техничких наука Универзитета у Новом Саду.

- [1] M. Fowler, *Domain-Specific Languages*. Addison-Wesley, 2011.
- [2] H. Bänder, “Decoupling Language and Editor – The Impact of the Language Server Protocol on Textual Domain-Specific Languages,” in *Proceedings of the 7th International Conference on Model-Driven Engineering and Software Development (MODELSWARD)*, 2019, pp. 129–140.
- [3] Microsoft, “Language Server Protocol Specification (Version 3.17).” Accessed: April 29, 2026. [Online]. Доступно на <https://microsoft.github.io/language-server-protocol>
- [4] D. Bork and P. Langer, “Language Server Protocol: An Introduction to the Protocol, Its Use, and Adoption for Web Modeling Tools,” *Enterprise Modelling and Information Systems Architectures (EMISAJ)*, vol. 18, p. 9:1–9:31, 2023.
- [5] R. Rodriguez-Echeverria, J. L. C. Izquierdo, M. Wimmer, and J. Cabot, “Towards a Language Server Protocol Infrastructure for Graphical Modeling,” in *Proceedings of the 21st ACM/IEEE International Conference on Model Driven Engineering Languages and Systems (MODELS)*, 2018, pp. 370–380.
- [6] M. Voelter, *DSL Engineering: Designing, Implementing and Using Domain-Specific Languages*. dslbook.org, 2013.
- [7] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Pearson, 2007.
- [8] T. Parr, *Language Implementation Patterns: Create Your Own Domain-specific and General Programming Languages*. in Pragmatic Bookshelf Series. Pragmatic Bookshelf, 2010. [Online]. Доступно на <https://books.google.rs/books?id=C7xuPgAACAAJ>
- [9] T. Ball, *Writing An Interpreter In Go*. Wainshain Press, 2016.
- [10] S. S. Muchnick, *Advanced Compiler Design and Implementation*. San Francisco, CA: Morgan Kaufmann, 1997.
- [11] The Rust Project Developers, “Guide to rustc Development.” Accessed: July 7, 2026. [Online]. Доступно на <https://rustc-dev-guide.rust-lang.org/>
- [12] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. in Addison-Wesley Professional Computing Series. Addison-Wesley, 1994.
- [13] R. Nystrom, *Crafting Interpreters*. Genever Benning, 2021. [Online]. Доступно на <https://craftinginterpreters.com/>
- [14] S. Furuhashi, “MessagePack Specification.” Accessed: July 7, 2026. [Online]. Доступно на <https://msgpack.org/>
- [15] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media, 2017.
- [16] A. M. Law, *Simulation Modeling and Analysis*, 5th ed. New York, NY: McGraw-Hill Education, 2014.
- [17] D. E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*, 3rd ed. Reading, MA: Addison-Wesley, 1997.
- [18] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes: The Art of Scientific Computing*, 3rd ed. New York, NY: Cambridge University Press, 2007.

-
- [19] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2016.
- [20] I. Dejanović, R. Vadera, G. Milosavljević, and Ž. Vuković, “textX: A Python tool for Domain-Specific Languages implementation,” *Knowledge-Based Systems*, vol. 115, pp. 1–4, 2017, doi: [10.1016/j.knosys.2016.10.023](https://doi.org/10.1016/j.knosys.2016.10.023).
- [21] B. Ford, “Parsing expression grammars: a recognition-based syntactic foundation,” in *Proceedings of the 31st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 2004, pp. 111–122. doi: [10.1145/964001.964011](https://doi.org/10.1145/964001.964011).
- [22] F. Campagne, *The MPS Language Workbench: Volume I*, 3rd ed. CreateSpace Independent Publishing Platform, 2016.
- [23] M. Voelter, J. Siegmund, T. Berger, and B. Kolb, “Towards user-friendly projectional editors,” *Software & Systems Modeling*, vol. 13, no. 4, pp. 1617–1645, 2014, doi: [10.1007/s10270-013-0329-0](https://doi.org/10.1007/s10270-013-0329-0).
- [24] A. Wright, H. Andrews, and B. Hutton, “JSON Schema: A Media Type for Describing JSON Data Formats,” technical report, 2022.
- [25] G. LLC, “Protocol Buffers Developer Guide.” 2026.
- [26] F. Contributors, “Faker JavaScript/Python Library Documentation.” 2026.
- [27] M. LLC, “Mockaroo: Realistic Data Generator and API Mocking Tool.” 2026.
- [28] J. E. Gentle, *Random Number Generation and Monte Carlo Methods*, 2nd ed. Springer Science & Business Media, 2003.
- [29] C. J. Date, *Database Design and Relational Theory: Normal Forms and All That Jazz*. O'Reilly Media, Inc., 2012.
- [30] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.
- [31] Rust Project Developers, “The Rust Standard Library Documentation.” 2026.
- [32] dani0595, “rmp-serde: A Rust MessagePack serializer/deserializer.” [Online]. Доступно на <https://github.com/3Hren/msgpack-rust>
- [33] K. B. Knapp and T. C. Community, “clap.” [Online]. Доступно на <https://docs.rs/clap>
- [34] J. Nielsen, *Usability Engineering*. Morgan Kaufmann, 1993.